Documentação do psutil

Links rápidos

- Página inicial
- Instalar
- Fórum
- Download
- Blog
- Contribuindo
- Guia de desenvolvimento
- O que há de novo

Sobre

psutil (utilitários de sistema e processos em Python) é uma biblioteca multiplataforma para obter informações sobre **processos** em execução e **utilização do sistema** (CPU, memória, discos, rede, sensores) em **Python** . É útil principalmente para **monitoramento do sistema** , **criação de perfis** , **limitação de recursos de processos** e **gerenciamento de processos em execução** . Implementa muitas funcionalidades oferecidas por ferramentas de linha de comando UNIX, como: *ps, top, lsof, netstat, ifconfig, who, df, kill, free, nice, ionice, iostat, iotop, uptime, pidof, tty, taskset, pmap* . Atualmente, o psutil suporta as seguintes plataformas:

- **Linux**
- **Windows**
- **macOS**
- **FreeBSD, OpenBSD** , **NetBSD**
- **Sol Solaris**
- **AIX**

As versões do Python suportadas são cPython 3.6+ e PyPy . A versão mais recente do psutil compatível com Python 2.7 é a psutil 6.1.1 .

A documentação do psutil que você está lendo é distribuída como uma única página HTML.

Patrocinadores

Sansec apivoid

Adicione seu logotipo

Financiamento

Embora o psutil seja um software livre e sempre será, o projeto se beneficiaria imensamente de algum financiamento. O psutil está entre os 100 pacotes Python mais baixados, e manter o controle de relatórios de bugs, o suporte ao usuário e a manutenção contínua tornou-se cada vez mais difícil para uma única pessoa. Se sua empresa utiliza o psutil de forma significativa, considere se tornar um patrocinador via GitHub , Open Collective ou PayPal . Os patrocinadores podem ter seu logotipo exibido aqui e na documentação do psutil .

Instalar

No Linux, Windows e macOS:

```
pip install psutil
```

Para outras plataformas, consulte as instruções de instalação mais detalhadas .

Funções relacionadas ao sistema

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

CPU

`psutil.cpu_times` (*percpu = False*)

Retorna os tempos de CPU do sistema como uma tupla nomeada. Cada atributo representa os segundos que a CPU passou no modo especificado. A disponibilidade dos atributos varia dependendo da plataforma:

- **usuário** : tempo gasto por processos normais em execução no modo usuário; no Linux, isso também inclui o tempo **do sistema operacional convidado.**
- **sistema** : tempo gasto por processos em execução no modo kernel
- **ocioso** : tempo gasto sem fazer nada

Campos específicos da plataforma:

- **nice** *(UNIX)* : tempo gasto por processos com prioridade (nice) em execução no modo de usuário; no Linux, isso também inclui o tempo **guest_nice.**
- **iowait** *(Linux)* : tempo gasto aguardando a conclusão de operações de E/S. Este tempo *não* é contabilizado no contador de tempo **ocioso** .
- **irq** *(Linux, BSD)* : tempo gasto no atendimento de interrupções de hardware
- **softirq** *(Linux)* : tempo gasto no atendimento de interrupções de software
- **steal** *(Linux 2.6.11+)* : tempo gasto por outros sistemas operacionais em execução em um ambiente virtualizado
- **convidado** *(Linux 2.6.24+)* : tempo gasto executando uma CPU virtual para sistemas operacionais convidados sob o controle do kernel do Linux.
- **guest_nice** *(Linux 3.2.0+)* : tempo gasto em execução de um convidado "nice" (CPU virtual para sistemas operacionais convidados sob o controle do kernel Linux)
- **interrupção** *(Windows)* : tempo gasto no atendimento de interrupções de hardware (similar a "irq" no UNIX)
- **dpc** *(Windows)* : tempo gasto no atendimento de chamadas de procedimento adiadas (DPCs); DPCs são interrupções que são executadas com prioridade mais baixa do que as interrupções padrão.

Quando `*percpu*` é `True` chamado, retorna uma lista de tuplas nomeadas para cada CPU lógica no sistema. O primeiro elemento da lista se refere à primeira CPU, o segundo elemento à segunda CPU e assim por diante. A ordem da lista é consistente entre as chamadas. Exemplo de saída no Linux:

```
>>> import psutil
>>> psutil.cpu_times()
scputimes(user=17411.7, nice=77.99, system=3797.02, idle=51266.57, iowait=732.58, irq=0.01, softirq=142.43, steal=0.0, guest=0.0, guest_nice=0.0)
```

Alterado na versão 4.1.0: adicionados os campos *interrupt* e *dpc* no Windows.

> **Aviso:** O tempo de CPU sempre deveria aumentar com o tempo, ou pelo menos permanecer o mesmo, pois o tempo não pode retroceder. Surpreendentemente, às vezes isso pode não acontecer (pelo menos no Windows e no Linux), veja #1210 .

`psutil.cpu_percent` (*interval = None , percpu = False*)

Retorna um número de ponto flutuante representando a utilização atual da CPU em todo o sistema, em porcentagem. Quando `*interval*` é maior que `true`, `0.0` compara os tempos de CPU do sistema decorridos antes e depois do intervalo (bloqueando). Quando `*interval*` é menor `0.0` ou `None` igual a `false`, compara os tempos de CPU do sistema decorridos desde a última chamada ou importação de módulo, retornando imediatamente. Isso significa que, na primeira vez que esta função for chamada, ela retornará um `0.0` valor sem significado, que deve ser ignorado. Nesse caso, recomenda-se, para maior precisão, que esta função seja chamada com pelo menos ` `0.1` false` segundos entre as chamadas. Quando `*percpu*` é `True` `true`, retorna uma lista de números de ponto flutuante representando a utilização em porcentagem para cada CPU. O primeiro elemento da lista refere-se à primeira CPU, o segundo elemento à segunda CPU e assim por diante. A ordem da lista é consistente entre as chamadas. Internamente, esta função mantém um mapa global (um dicionário) onde cada chave é o ID da thread que a chamou (` threading.get_ident`). Isso significa que ela pode ser chamada de threads diferentes, em intervalos diferentes, e ainda retornar resultados significativos e independentes.

```
>>> import psutil
>>> # blocking
>>> psutil.cpu_percent(interval=1)
2.0
>>> # non-blocking (percentage since last call)
>>> psutil.cpu_percent(interval=None)
2.9
>>> # blocking, per-cpu
>>> psutil.cpu_percent(interval=1, percpu=True)
[2.0, 1.0]
>>>
```

> **Aviso:** Na primeira vez que esta função for chamada com `*interval* = `0.0` or`, `None` ela retornará um `0.0` valor sem significado que você deve ignorar.

Alterado na versão 5.9.6: a função agora é thread-safe.

`psutil.cpu_times_percent` (*interval = None , percpu = False*)

Semelhante a, `cpu_percent()` mas fornece porcentagens de utilização para cada tempo de CPU específico, conforme retornado por `psutil.cpu_times(percpu=True)` . Os argumentos `*interval*` e `*percpu*` têm o mesmo significado que em `cpu_percent()` . No Linux, as porcentagens de "guest" e "guest_nice" não são contabilizadas nas porcentagens de "user" e "user_nice".

> **Aviso:** Na primeira vez que esta função for chamada com `*interval* = `0.0` or`, `None` ela retornará um `0.0` valor sem significado que você deve ignorar.

Alterado na versão 4.1.0: dois novos campos *de interrupção* e *DPC* são retornados no Windows.

Alterado na versão 5.9.6: a função agora é thread-safe.

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Retorna o número de CPUs lógicas no sistema (similar a `os.cpu_count`) ou `None` `NULL` se indeterminado. Diferentemente de `os.cpu_count` , este valor não é influenciado pela `PYTHON_CPU_COUNT` variável de ambiente introduzida no Python 3.13. "CPUs lógicas" significa o número de núcleos físicos multiplicado pelo número de threads que podem ser executadas em cada núcleo (conhecido como Hyper-Threading). É o que os provedores de nuvem geralmente chamam de vCPUs. Se o número de CPUs *lógicas* for verdadeiro, `False` retorna apenas o número de núcleos físicos ou ` `None` NULL` se indeterminado. No OpenBSD e NetBSD, `psutil.cpu_count(logical=False)` sempre retorna ` `None` NULL`. Exemplo em um sistema com 2 núcleos + Hyper-Threading:

```
>>> import psutil
>>> psutil.cpu_count()
4
>>> psutil.cpu_count(logical=False)
2
```

Note que esse valor `psutil.cpu_count()` pode não ser necessariamente equivalente ao número real de CPUs que o processo atual pode usar. Isso pode variar caso a afinidade de CPU do processo tenha sido alterada, se grupos de CPU (cgroups) do Linux estiverem sendo usados ou (no caso do Windows) em sistemas que utilizam grupos de processadores ou que possuem mais de 64 CPUs. O número de CPUs utilizáveis pode ser obtido com:

```
>>> len(psutil.Process().cpu_affinity())
1
```

psutil. **cpu_stats** ( )

Retorna várias estatísticas da CPU como uma tupla nomeada:

- **ctx_switches** : número de trocas de contexto (voluntárias + involuntárias) desde a inicialização.
- **interrupções** : número de interrupções desde a inicialização.
- **soft_interrupts** : número de interrupções de software desde a inicialização. Sempre definido como verdadeiro `0` no Windows e no SunOS.
- **syscalls** : número de chamadas de sistema desde a inicialização. Sempre definido como verdadeiro `0` no Linux.

Exemplo (Linux):

```
>>> import psutil
>>> psutil.cpu_stats()
scpustats(ctx_switches=20455687, interrupts=6598984, soft_interrupts=2134212, syscalls=0)
```

Adicionado na versão 4.1.0.

psutil. **cpu_freq** (*percpu = False*)

Retorna a frequência da CPU como uma tupla nomeada incluindo as frequências *atual* , *mínima* e *máxima*, expressas em MHz. No Linux, a frequência *atual* representa o valor em tempo real; em todas as outras plataformas, geralmente representa o valor nominal "fixo" (que nunca muda). Se `*percpu*` for verdadeiro `True` e o sistema suportar a recuperação da frequência por CPU (Linux e FreeBSD), uma lista de frequências será retornada para cada CPU; caso contrário, uma lista com um único elemento será retornada. Se *os valores mínimo* e *máximo* não puderem ser determinados, serão definidos como ` null` `0.0` .

Exemplo (Linux):

```
>>> import psutil
>>> psutil.cpu_freq()
scpufreq(current=931.42925, min=800.0, max=3500.0)
>>> psutil.cpu_freq(percpu=True)
[scpufreq(current=2394.945, min=800.0, max=3500.0),
 scpufreq(current=2236.812, min=800.0, max=3500.0),
 scpufreq(current=1703.609, min=800.0, max=3500.0),
 scpufreq(current=1754.289, min=800.0, max=3500.0)]
```

Disponibilidade: Linux, macOS, Windows, FreeBSD, OpenBSD. *O processamento por CPU (percpu)* é suportado apenas em Linux e FreeBSD.

Adicionado na versão 5.1.0.

Alterado na versão 5.5.1: adicionado suporte ao FreeBSD.

Alterado na versão 5.9.1: adicionado suporte ao OpenBSD.

psutil.getloadavg ( )

Retorna a carga média do sistema nos últimos 1, 5 e 15 minutos como uma tupla. O parâmetro "load" representa os processos que estão em estado executável, seja utilizando a CPU ou aguardando para utilizá-la (por exemplo, aguardando E/S de disco). Em sistemas UNIX, isso depende da função `os.getloadavg` . No Windows, isso é emulado por meio de uma API do Windows que cria uma thread que permanece em execução em segundo plano e atualiza os resultados a cada 5 segundos, imitando o comportamento do UNIX. Portanto, no Windows, na primeira vez que essa função for chamada e nos 5 segundos seguintes, ela retornará uma tupla sem significado. Os números retornados só fazem sentido se relacionados ao número de núcleos de CPU instalados no sistema. Assim, por exemplo, um valor de *3,14* em um sistema com 10 CPUs lógicas significa que a carga do sistema foi de 31,4% nos últimos N minutos. `(0.0, 0.0, 0.0)`

```
>>> import psutil
>>> psutil.getloadavg()
(3.14, 3.89, 4.67)
>>> psutil.cpu_count()
10
>>> # percentage representation
>>> [x / psutil.cpu_count() * 100 for x in psutil.getloadavg()]
```

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Disponibilidade: Unix, Windows

Adicionado na versão 5.6.2.

Memória

`psutil.virtual_memory ( )`

Retorna estatísticas sobre o uso da memória do sistema como uma tupla nomeada, incluindo os seguintes campos, expressos em bytes.

Principais métricas:

- **total** : memória física total (excluindo swap).
- **Disponível** : a memória que pode ser alocada instantaneamente aos processos sem que o sistema recorra à memória de troca (swap). Esse valor é calculado somando-se diferentes métricas de memória que variam de acordo com a plataforma. Sua função é monitorar o uso real de memória de forma multiplataforma.
- **porcentagem** : a porcentagem de uso calculada como . `(total - available) / total * 100`

Outras métricas:

- **usado** : memória utilizada, calculada de forma diferente dependendo da plataforma e destinada apenas a fins informativos. **Total - livre** não corresponde necessariamente **ao usado** .
- **livre** : memória não utilizada (zerada) que está prontamente disponível; observe que isso não reflete a memória realmente disponível (use **disponível** em vez disso). **total - usado** não corresponde necessariamente **a livre** .
- **Ativo** *(UNIX)* : memória atualmente em uso ou usada muito recentemente, e por isso está na RAM.
- **inativo** *(UNIX)* : memória marcada como não utilizada.
- **buffers** *(Linux, BSD)* : cache para itens como metadados do sistema de arquivos.
- **cached** *(Linux, BSD)* : cache para várias coisas.
- **memória compartilhada** *(Linux, BSD)* : memória que pode ser acessada simultaneamente por múltiplos processos.
- **slab** *(Linux)* : cache de estruturas de dados no kernel.
- **Wired** *(BSD, macOS)* : memória marcada para permanecer sempre na RAM. Ela nunca é movida para o disco.

A soma de **usado** e **disponível** não é necessariamente igual **ao total** . No Windows , **disponível** e **livre** são iguais. Veja o script meminfo.py, que fornece um exemplo de como converter bytes em um formato legível para humanos.

> **Observação:** Se você apenas deseja saber quanta memória física resta em diferentes plataformas, basta usar os campos **"disponível"** e **"em porcentagem"** .

```
>>> import psutil
>>> mem = psutil.virtual_memory()
>>> mem
svmem(total=10367352832, available=6472179712, percent=37.6, used=8186245120, free=2181107712, active=4748992512, inactive=2758115328, buffers=790724608,
cached=3500347392, shared=787554304, slab=199348224)
>>>
>>> THRESHOLD = 100 * 1024 * 1024  # 100MB
>>> if mem.available <= THRESHOLD:
...     print("warning")
...
>>>
```

Alterado na versão 4.2.0: adicionada métrica *compartilhada no Linux.*

Alterado na versão 5.4.4: adicionada a métrica *slab* no Linux.

`psutil.swap_memory ( )`

Retorna as estatísticas de memória swap do sistema como uma tupla nomeada, incluindo os seguintes campos:

- **total** : total de memória swap em bytes
- **usado** : memória de troca usada em bytes
- **livre** : memória swap livre em bytes
- **porcentagem** : a porcentagem de utilização calculada como `(total - available) / total * 100`
- **sin** : o número de bytes que o sistema trocou do disco para a memória (acumulado)
- **sout** : o número de bytes que o sistema trocou do disco (acumulado)

No Windows, os parâmetros sin e **sout** são sempre definidos como `0`. Consulte o script meminfo.py, que fornece um exemplo de como converter bytes em um formato legível para humanos.

```
>>> import psutil
>>> psutil.swap_memory()
sswap(total=2097147904L, used=886620160L, free=1210527744L, percent=42.3, sin=1050411008, sout=1906720768)
```

Alterado na versão 5.2.3: no Linux, esta função depende do sistema de arquivos /proc em vez da chamada de sistema sysinfo(), para que possa ser usada em conjunto com outras funções `psutil.PROCFS_PATH` a fim de recuperar informações de memória sobre contêineres Linux, como Docker e Heroku.

Discos

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Retorna todas as partições de disco montadas como uma lista de tuplas nomeadas, incluindo dispositivo, ponto de montagem e tipo de sistema de arquivos, similar ao comando "df" no UNIX. Se o parâmetro `*all*` `False` for verdadeiro, o comando tenta distinguir e retornar apenas dispositivos físicos (por exemplo, discos rígidos, unidades de CD-ROM, pen drives) e ignora todos os outros (por exemplo, sistemas de arquivos pseudo, em memória, duplicados e inacessíveis). Observe que isso pode não ser totalmente confiável em todos os sistemas (por exemplo, no BSD, esse parâmetro é ignorado). Consulte o script `disk_usage.py` para obter um exemplo de uso. Retorna uma lista de tuplas nomeadas com os seguintes campos:

- **dispositivo** : o caminho do dispositivo (ex.: `"/dev/hda1"`). No Windows, é a letra da unidade (ex.: `"C:\\"`).
- **mountpoint** : o caminho do ponto de montagem (ex: `"/"`). No Windows, é a letra da unidade (ex: `"C:\\"`).
- **fstype** : o sistema de arquivos da partição (por exemplo, `"ext3"` no UNIX ou `"NTFS"` no Windows).
- **`opts`** : uma sequência de caracteres separados por vírgulas que indica diferentes opções de montagem para a unidade/partição. Depende da plataforma.

```
>>> import psutil
>>> psutil.disk_partitions()
[sdiskpart(device='/dev/sda3', mountpoint='/', fstype='ext4', opts='rw,errors=remount-ro'),
 sdiskpart(device='/dev/sda7', mountpoint='/home', fstype='ext4', opts='rw')]
```

Alterado na versão 5.7.4: adicionados os campos *maxfile* e *maxpath*.

Alterado na versão 6.0.0: campos *maxfile* e *maxpath* removidos.

`psutil.disk_usage` (*caminho*)

Retorna estatísticas de uso de disco da partição que contém o *caminho* fornecido , como uma tupla nomeada, incluindo espaço **total** , **usado** e **livre expressos em bytes, além da porcentagem** de uso. `OSError` Uma exceção será lançada se *o caminho* não existir. A partir do Python 3.3, esta função também está disponível como `shutil.disk_usage` (consulte BPO-12442). Veja o script `disk_usage.py` que fornece um exemplo de uso.

```
>>> import psutil
>>> psutil.disk_usage('/')
sdiskusage(total=21378641920, used=4809781248, free=15482871808, percent=22.5)
```

Observação: O UNIX geralmente reserva 5% do espaço total em disco para o usuário root. Os campos `*total*` e `*used*` no UNIX referem-se ao espaço total e ao espaço usado, enquanto `*free*` representa o espaço disponível para o **usuário** e `*percent*` representa a utilização **pelo usuário (veja** o código-fonte). É por isso que o valor *em porcentagem* pode parecer 5% maior do que o esperado. Observe também que ambos os 4 valores correspondem ao utilitário de linha de comando "df".

Alterado na versão 4.3.0: o valor percentual leva em consideração o espaço reservado na raiz.

`psutil.` **`disk_io_counters`** (*perdisk = False , nowrap = True*)

Retorna as estatísticas de E/S de disco de todo o sistema como uma tupla nomeada, incluindo os seguintes campos:

- **read_count** : número de leituras
- **write_count** : número de gravações
- **read_bytes** : número de bytes lidos
- **write_bytes** : número de bytes escritos

Campos específicos da plataforma:

- **read_time** : (exceto *NetBSD* e *OpenBSD*) tempo gasto na leitura do disco (em milissegundos)
- **write_time** : (exceto *NetBSD* e *OpenBSD*) tempo gasto gravando em disco (em milissegundos)
- **busy_time** : (*Linux* , *FreeBSD*) tempo gasto realizando operações de E/S (em milissegundos)
- **read_merged_count** (*Linux*): número de leituras mescladas (consulte a documentação do iostats)
- **write_merged_count** (*Linux*): número de gravações mescladas (consulte a documentação do iostats)

Se `*perdisk*` for verdadeiro, `True` retorne as mesmas informações para cada disco físico instalado no sistema como um dicionário com os nomes das partições como chaves e a tupla nomeada descrita acima como valores. Veja `iotop.py` para um exemplo de aplicação. Em alguns sistemas, como o Linux, em sistemas muito ocupados ou de longa duração, os números retornados pelo kernel podem sofrer estouro e reiniciar (recomeçar do zero). Se `*nowrap*` for verdadeiro, `True` o `psutil` detectará e ajustará esses números entre as chamadas de função, adicionando o "valor antigo" ao "valor novo" para que os números retornados estejam sempre aumentando ou permaneçam os mesmos, mas nunca diminuam. `disk_io_counters.cache_clear()` Pode ser usado para invalidar o cache `*nowrap*` . No Windows, pode ser necessário executar o comando `psutil -f` no `cmd.exe` primeiro para habilitar os contadores de E/S. Em máquinas sem disco, esta função retornará `true` se `*perdisk*` for verdadeiro . `diskperf -y` `None` `{}` `True`

```
>>> import psutil
>>> psutil.disk_io_counters()
sdiskio(read_count=8141, write_count=2431, read_bytes=290203, write_bytes=537676, read_time=5868, write_time=94922)
>>>
>>> psutil.disk_io_counters(perdisk=True)
{'sda1': sdiskio(read_count=920, write_count=1, read_bytes=2933248, write_bytes=512, read_time=6016, write_time=4),
 'sda2': sdiskio(read_count=18707, write_count=8830, read_bytes=6060, write_bytes=3443, read_time=24585, write_time=1572),
 'sdb1': sdiskio(read_count=161, write_count=0, read_bytes=786432, write_bytes=0, read_time=44, write_time=0)}
```

Observação: No Windows, pode ser necessário executar o comando primeiro, caso contrário, esta função não encontrará nenhum disco. `"diskperf -y"`

Alterado na versão 5.3.0: os números não reiniciam mais (não começam do zero) entre chamadas graças ao novo argumento *nowrap* .

Alterações na versão 4.0.0: adicionados os campos *busy_time* (Linux, FreeBSD), *read_merged_count* e *write_merged_count* (Linux).

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Rede

`psutil.net_io_counters` (*pernic = False , nowrap = True*)

Retorna as estatísticas de E/S de rede de todo o sistema como uma tupla nomeada, incluindo os seguintes atributos:

- **bytes_sent** : número de bytes enviados
- **bytes_recv** : número de bytes recebidos
- **pacotes_enviados** : número de pacotes enviados
- **pacotes_recebidos** : número de pacotes recebidos
- **errin** : número total de erros durante o recebimento
- **erro** : número total de erros durante o envio
- **dropin** : número total de pacotes recebidos que foram descartados
- **dropout** : número total de pacotes de saída que foram descartados (sempre 0 no macOS e BSD)

Se `*pernic*` estiver definido, `True` retorne as mesmas informações para cada interface de rede instalada no sistema como um dicionário com os nomes das interfaces de rede como chaves e a tupla nomeada descrita acima como valores. Em alguns sistemas, como o Linux, em sistemas muito ocupados ou de longa duração, os números retornados pelo kernel podem sofrer estouro e reiniciar (recomeçar do zero). Se ` *nowrap*` estiver definido `True`, o `psutil` detectará e ajustará esses números entre as chamadas de função, adicionando o "valor antigo" ao "valor novo" para que os números retornados estejam sempre aumentando ou permaneçam os mesmos, mas nunca diminuam. `net_io_counters.cache_clear()` Pode ser usado para invalidar o cache `*nowrap*` . Em máquinas sem interfaces de rede, esta função retornará `null` `None` ou ` false` `{}` se `*pernic*` estiver definido `True`.

```
>>> import psutil
>>> psutil.net_io_counters()
snetio(bytes_sent=14508483, bytes_recv=62749361, packets_sent=84311, packets_recv=94888, errin=0, errout=0, dropin=0, dropout=0)
>>>
>>> psutil.net_io_counters(pernic=True)
{'lo': snetio(bytes_sent=547971, bytes_recv=547971, packets_sent=5075, packets_recv=5075, errin=0, errout=0, dropin=0, dropout=0),
'wlan0': snetio(bytes_sent=13921765, bytes_recv=62162574, packets_sent=79097, packets_recv=89648, errin=0, errout=0, dropin=0, dropout=0)}
```

Veja também nettop.py e ifconfig.py para um exemplo de aplicação.

Alterado na versão 5.3.0: os números não reiniciam mais (não começam do zero) entre chamadas graças ao novo argumento *nowrap* .

`psutil.net_connections` (*kind = 'inet '*)

Retorna as conexões de soquete de todo o sistema como uma lista de tuplas nomeadas. Cada tupla nomeada fornece 7 atributos:

- **fd** : o descritor de arquivo do socket. Se a conexão se referir ao processo atual, ele poderá ser passado para socket.fromfd para obter um objeto de socket utilizável. No Windows e no SunOS, este valor é sempre definido como `-1`.
- **família** : a família de endereços, seja AF_INET , AF_INET6 ou AF_UNIX .
- **tipo** : o tipo de endereço, seja SOCK_STREAM , SOCK_DGRAM ou SOCK_SEQPACKET .
- **laddr** : o endereço local como uma tupla nomeada ou um valor no caso de sockets AF_UNIX. Para sockets UNIX, consulte as notas abaixo. `(ip, port)` `path`
- **raddr** : o endereço remoto como uma tupla nomeada ou um valor absoluto no caso de sockets UNIX. Quando o endpoint remoto não estiver conectado, você receberá uma tupla vazia (AF_INET*) ou (AF_UNIX). Para sockets UNIX, consulte as notas abaixo. `(ip, port)` `path` `""`
- **status** : representa o status de uma conexão TCP. O valor de retorno é uma das constantes psutil.CONN_* (uma string). Para sockets UDP e UNIX, será sempre `psutil.CONN_NONE`.
- **pid** : o PID do processo que abriu o socket, se recuperável, caso contrário `None`, . Em algumas plataformas (ex.: Linux), a disponibilidade deste campo varia dependendo dos privilégios do processo (é necessário privilégio de root).

O parâmetro `*kind*` é uma string que filtra as conexões que correspondem aos seguintes critérios:

Valor de tipo	Conexões usando
`"inet"`	IPv4 e IPv6
`"inet4"`	IPv4
`"inet6"`	IPv6
`"tcp"`	TCP
`"tcp4"`	TCP sobre IPv4
`"tcp6"`	TCP sobre IPv6
`"udp"`	UDP
`"udp4"`	UDP sobre IPv4
`"udp6"`	UDP sobre IPv6
`"unix"`	Socket UNIX (protocolos UDP e TCP)
`"all"`	a soma de todas as famílias e protocolos possíveis

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

No macOS e no AIX, esta função requer privilégios de root. Para obter conexões por processo, use `Process.net_connections()`. Veja também o script de exemplo netstat.py . Exemplo:

```
>>> import psutil
>>> psutil.net_connections()
[pconn(fd=115, family=<AddressFamily.AF_INET: 2>, type=<SocketType.SOCK_STREAM: 1>, laddr=addr(ip='10.0.0.1', port=48776), raddr=addr(ip='93.186.135.91',
port=80), status='ESTABLISHED', pid=1254),
 pconn(fd=117, family=<AddressFamily.AF_INET: 2>, type=<SocketType.SOCK_STREAM: 1>, laddr=addr(ip='10.0.0.1', port=43761), raddr=addr(ip='72.14.234.100',
port=80), status='CLOSING', pid=2987),
 pconn(fd=-1, family=<AddressFamily.AF_INET: 2>, type=<SocketType.SOCK_STREAM: 1>, laddr=addr(ip='10.0.0.1', port=60759), raddr=addr(ip='72.14.234.104',
port=80), status='ESTABLISHED', pid=None),
 pconn(fd=-1, family=<AddressFamily.AF_INET: 2>, type=<SocketType.SOCK_STREAM: 1>, laddr=addr(ip='10.0.0.1', port=51314), raddr=addr(ip='72.14.234.83',
port=443), status='SYN_SENT', pid=None)
 ...]
```

Aviso: No Linux, a recuperação de algumas conexões requer privilégios de root. Se o psutil não for executado como root, essas conexões serão ignoradas silenciosamente em vez de gerar uma exceção `PermissionError`. Isso significa que a lista retornada pode estar incompleta.

Observação: (macOS e AIX) `psutil.AccessDenied` é sempre levantado, a menos que esteja sendo executado como root. Esta é uma limitação do sistema operacional e `lsof` faz o mesmo.

Observação: (Solaris) Sockets UNIX não são suportados.

Observação: (Linux, FreeBSD, OpenBSD) O campo *raddr* para sockets UNIX é sempre definido como `""` (string vazia). Esta é uma limitação do sistema operacional.

Adicionado na versão 2.1.0.

Alterado na versão 5.3.0: o socket "fd" agora é definido de forma real em vez de ser `-1`.

Alterado na versão 5.3.0: laddr *e* raddr *agora* são tuplas nomeadas.

Alterado na versão 5.9.5: : OpenBSD: recupera o caminho *laddr* para sockets AF_UNIX (antes era uma string vazia).

`psutil.net_if_addrs ( )`

Retorna os endereços associados a cada NIC (placa de interface de rede) instalada no sistema como um dicionário cujas chaves são os nomes das NICs e o valor é uma lista de tuplas nomeadas para cada endereço atribuído à NIC. Cada tupla nomeada inclui 5 campos:

- **família** : a família de endereços, AF_INET ou AF_INET6 ou `psutil.AF_LINK`, que se refere a um endereço MAC.
- **Endereço** : o endereço da placa de rede principal (sempre definido).
- **netmask** : o endereço de máscara de rede (pode ser `None`).
- **transmissão** : o endereço de transmissão (pode ser `None`).
- **PTP** : significa "ponto a ponto"; é o endereço de destino em uma interface ponto a ponto (normalmente uma VPN). *Broadcast* e *PTP* são mutuamente exclusivos. Talvez `None`...

Exemplo:

```
>>> import psutil
>>> psutil.net_if_addrs()
{'lo': [snicaddr(family=<AddressFamily.AF_INET: 2>, address='127.0.0.1', netmask='255.0.0.0', broadcast='127.0.0.1', ptp=None),
        snicaddr(family=<AddressFamily.AF_INET6: 10>, address='::1', netmask='ffff:ffff:ffff:ffff:ffff:ffff:ffff:ffff', broadcast=None, ptp=None),
        snicaddr(family=<AddressFamily.AF_LINK: 17>, address='00:00:00:00:00:00', netmask=None, broadcast='00:00:00:00:00:00', ptp=None)],
 'wlan0': [snicaddr(family=<AddressFamily.AF_INET: 2>, address='192.168.1.3', netmask='255.255.255.0', broadcast='192.168.1.255', ptp=None),
           snicaddr(family=<AddressFamily.AF_INET6: 10>, address='fe80::c685:8ff:fe45:641%wlan0', netmask='ffff:ffff:ffff:ffff::', broadcast=None, ptp=None),
           snicaddr(family=<AddressFamily.AF_LINK: 17>, address='c4:85:08:45:06:41', netmask=None, broadcast='ff:ff:ff:ff:ff:ff', ptp=None)]}
>>>
```

Veja também nettop.py e ifconfig.py para um exemplo de aplicação.

Observação: Se você estiver interessado em outras famílias de interfaces (por exemplo, AF_BLUETOOTH), pode usar a extensão netifaces, que é mais robusta.

Observação: Você pode ter mais de um endereço da mesma família associado a cada interface (é por isso que os valores de dicionário são listas).

Observação: *Os métodos broadcast* e *ptp* não são suportados no Windows e estão sempre em modo de espera `None`.

Adicionado na versão 3.0.0.

Alterado na versão 3.2.0: foi adicionado o campo *ptp* .

Alterado na versão 4.4.0: adicionado suporte para o campo *netmask* no Windows, que não é mais `None`.

Alterado na versão 7.0.0: adicionado suporte para campo *de transmissão* no Windows, que não é mais `None`.

`psutil.net_if_stats ( )`

Retorna informações sobre cada NIC (placa de interface de rede) instalada no sistema como um dicionário cujas chaves são os nomes das NICs e o valor é uma tupla nomeada com os seguintes campos:

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

- **velocidade** : a velocidade da NIC expressa em megabits (MB), se não puder ser determinada (ex: 'localhost') será definida como `0`.
- **mtu** : Unidade máxima de transmissão da placa de rede, expressa em bytes.
- **flags** : uma sequência de flags separadas por vírgulas na interface (pode ser uma sequência vazia). As flags possíveis são: `up`, `broadcast`, `debug`, `loopback`, `pointopoint`, `notrailers`, `running`, `noarp`, `promisc`, `allmulti`, `master`, `slave`, `multicast`, `portsel`, `dynamic`, `oactive`, `simplex`, `link0`, `link1`, `link2`, e `d2` (algumas flags estão disponíveis apenas em determinadas plataformas).

  Disponibilidade: UNIX

Exemplo:

```
>>> import psutil
>>> psutil.net_if_stats()
{'eth0': snicstats(isup=True, duplex=<NicDuplex.NIC_DUPLEX_FULL: 2>, speed=100, mtu=1500, flags='up,broadcast,running,multicast'),
 'lo': snicstats(isup=True, duplex=<NicDuplex.NIC_DUPLEX_UNKNOWN: 0>, speed=0, mtu=65536, flags='up,loopback,running')}
```

Veja também nettop.py e ifconfig.py para um exemplo de aplicação.

Adicionado na versão 3.0.0.

Alterado na versão 5.7.3: o comando isup no UNIX também verifica se a placa de rede está em funcionamento.

Alterado na versão 5.9.3: foi adicionado o campo *flags no POSIX.*

Sensores

`psutil.sensores_temperaturas(fahrenheit = Falso)`

Retorna as temperaturas do hardware. Cada entrada é uma tupla nomeada que representa um determinado sensor de temperatura de hardware (pode ser uma CPU, um disco rígido ou outro componente, dependendo do sistema operacional e de sua configuração). Todas as temperaturas são expressas em Celsius, a menos que *Fahrenheit* esteja definido como `True`. Se os sensores não forem suportados pelo sistema operacional, um dicionário vazio será retornado. Exemplo:

```
>>> import psutil
>>> psutil.sensors_temperatures()
{'acpitz': [shwtemp(label='', current=47.0, high=103.0, critical=103.0)],
 'asus': [shwtemp(label='', current=47.0, high=None, critical=None)],
 'coretemp': [shwtemp(label='Physical id 0', current=52.0, high=100.0, critical=100.0),
              shwtemp(label='Core 0', current=45.0, high=100.0, critical=100.0),
              shwtemp(label='Core 1', current=52.0, high=100.0, critical=100.0),
              shwtemp(label='Core 2', current=45.0, high=100.0, critical=100.0),
              shwtemp(label='Core 3', current=47.0, high=100.0, critical=100.0)]}
```

Veja também temperatures.py e sensors.py para um exemplo de aplicação.

Disponibilidade: Linux, FreeBSD

Adicionado na versão 5.1.0.

Alterado na versão 5.5.0: adicionado suporte ao FreeBSD

`psutil.sensores_ventiladores()`

Retorna a velocidade das ventoinhas do hardware. Cada entrada é uma tupla nomeada que representa uma determinada ventoinha com sensor de hardware. A velocidade da ventoinha é expressa em RPM (rotações por minuto). Se os sensores não forem suportados pelo sistema operacional, um dicionário vazio será retornado. Exemplo:

```
>>> import psutil
>>> psutil.sensors_fans()
{'asus': [sfan(label='cpu_fan', current=3200)]}
```

Veja também fans.py e sensors.py para um exemplo de aplicação.

Disponibilidade: Linux

Adicionado na versão 5.2.0.

`psutil.sensores_bateria()`

Retorna informações sobre o status da bateria como uma tupla nomeada, incluindo os seguintes valores. Se nenhuma bateria estiver instalada ou se as métricas não puderem ser determinadas, `None` será retornado o valor padrão.

- **porcentagem** : carga restante da bateria em porcentagem.
- **secsleft** : uma estimativa aproximada de quantos segundos restam antes que a bateria se esgote. Se o cabo de alimentação CA estiver conectado, este valor é definido como `psutil.POWER_TIME_UNLIMITED`. Se não for possível determinar, é definido como `psutil.POWER_TIME_UNKNOWN`.
- **power_plugged** : `True` indica se o cabo de alimentação CA está conectado; `False` caso contrário, ou `None` se não for possível determinar.

Exemplo:

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

```
>>> import psutil
>>>
>>> def secs2hours(secs):
...     mm, ss = divmod(secs, 60)
...     hh, mm = divmod(mm, 60)
...     return "%d:%02d:%02d" % (hh, mm, ss)
...
>>> battery = psutil.sensors_battery()
>>> battery
sbattery(percent=93, secsleft=16628, power_plugged=False)
>>> print("charge = %s%%, time left = %s" % (battery.percent, secs2hours(battery.secsleft)))
charge = 93%, time left = 4:37:08
```

Veja também battery.py e sensors.py para um exemplo de aplicação.

Disponibilidade: Linux, Windows, FreeBSD

Adicionado na versão 5.1.0.

Alterado na versão 5.4.2: adicionado suporte para macOS

Outras informações do sistema

`psutil.boot_time ( )`

Retorna o tempo de inicialização do sistema expresso em segundos desde a época Unix (segundos desde 1º de janeiro de 1970, à meia-noite UTC). O valor retornado é baseado no relógio do sistema, o que significa que pode ser afetado por alterações como ajustes manuais ou sincronização de tempo (por exemplo, NTP).

```
>>> import psutil, datetime
>>> psutil.boot_time()
1389563460.0
>>> datetime.datetime.fromtimestamp(psutil.boot_time()).strftime("%Y-%m-%d %H:%M:%S")
'2014-01-12 22:51:00'
```

Observação: No Windows, esta função pode retornar um tempo com uma diferença de 1 segundo se for usada em processos diferentes (veja a questão nº 1007).

`psutil. usuários ( )`

Retorna os usuários atualmente conectados ao sistema como uma lista de tuplas nomeadas, incluindo os seguintes campos:

- **nome** : o nome do usuário.
- **terminal** : o tty ou pseudo-tty associado ao usuário, se houver, caso contrário `None` .
- **host** : o nome do host associado à entrada, se houver.
- **started** : o horário de criação como um número de ponto flutuante expresso em segundos desde a época.
- **pid** : o PID do processo de login (como sshd, tmux, gdm-session-worker, ...). No Windows e no OpenBSD, este valor é sempre definido como `None` .

Exemplo:

```
>>> import psutil
>>> psutil.users()
[suser(name='giampaolo', terminal='pts/2', host='localhost', started=1340737536.0, pid=1352),
 suser(name='giampaolo', terminal='pts/3', host='localhost', started=1340737792.0, pid=1788)]
```

Alterado na versão 5.3.0: adicionado o campo “pid”

Processos

Funções

`psutil. pids ( )`

`process_iter()` Retorna uma lista ordenada dos PIDs em execução. É preferível iterar sobre todos os processos para evitar condições de corrida .

```
>>> import psutil
>>> psutil.pids()
[1, 2, 3, 5, 7, 8, 9, 10, 11, 12, 13, 14, 15, 17, 18, 19, ..., 32498]
```

Alterado na versão 5.6.0: os PIDs são retornados em ordem classificada.

`psutil.process_iter ( attrs = None , ad_value = None )`

Retorna um iterador que produz uma `Process` instância de classe para todos os processos em execução na máquina local. Isso deve ser preferido em relação `psutil.pids()` à iteração sobre os processos, pois a recuperação de informações é segura contra condições de corrida.

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

`attrs` e `ad_value` têm o mesmo significado que em ` `Process.as_dict()`.`. Se `attrs` for especificado `Process.as_dict()`, o resultado será armazenado como um `info` atributo anexado às `Process` instâncias retornadas. Se `attrs` for uma lista vazia, todas as informações do processo serão recuperadas (mais lento).

A ordem de classificação em que os processos são retornados é baseada em seu PID.

Exemplo:

```
>>> import psutil
>>> for proc in psutil.process_iter(['pid', 'name', 'username']):
...     print(proc.info)
...
{'name': 'systemd', 'pid': 1, 'username': 'root'}
{'name': 'kthreadd', 'pid': 2, 'username': 'root'}
{'name': 'ksoftirqd/0', 'pid': 3, 'username': 'root'}
...
```

Compreensões de dicionário para criar uma estrutura de dados: `{pid: info, ...}`

```
>>> import psutil
>>> procs = {p.pid: p.info for p in psutil.process_iter(['name', 'username'])}
>>> procs
{1: {'name': 'systemd', 'username': 'root'},
 2: {'name': 'kthreadd', 'username': 'root'},
 3: {'name': 'ksoftirqd/0', 'username': 'root'},
 ...}
```

Limpar cache interno:

```
>>> psutil.process_iter.cache_clear()
```

Alterado na versão 5.3.0: adicionados os parâmetros "attrs" e "ad_value".

Alterado na versão 6.0.0: não verifica mais se cada PID de processo retornado foi reutilizado.

Alterado na versão 6.0.0: `psutil.process_iter.cache_clear()` API adicionada .

`psutil.pid_exists` (*pid*)

Verifique se o PID fornecido existe na lista de processos atual. Isso é mais rápido do que fazer e deve ser preferido. `pid in psutil.pids()`

`psutil.wait_procs` (*procs* , *timeout = None* , *callback = None*)

`Process` Função auxiliar que aguarda a finalização de uma lista de instâncias. Retorna uma tupla indicando quais processos foram finalizados e quais ainda estão ativos. Os processos *finalizados* terão um novo atributo `returncode` indicando o status de saída do processo, conforme retornado por ` . é uma função que é chamada quando um dos processos aguardados é finalizado e uma instância é passada como argumento de retorno de chamada (a instância também terá um atributo `returncode` definido). Esta função retornará assim que todos os processos forem finalizados ou quando *o tempo limite* (em segundos) for atingido. Diferentemente de `, ela não lançará uma exceção se o tempo limite for atingido. Um caso de uso típico pode ser: `(gone, alive)` `Process.wait()` `callback` `Process` `Process.wait()` `TimeoutExpired`

- Enviar sinal SIGTERM para uma lista de processos.
- Dê-lhes algum tempo para terminar.
- Envie SIGKILL para aqueles que ainda estão vivos.

Exemplo que encerra e aguarda todos os filhos deste processo:

```
import psutil

def on_terminate(proc):
    print("process {} terminated with exit code {}".format(proc, proc.returncode))

procs = psutil.Process().children()
for p in procs:
    p.terminate()
gone, alive = psutil.wait_procs(procs, timeout=3, callback=on_terminate)
for p in alive:
    p.kill()
```

Exceções

aula `psutil.` **`Erro`**

Classe base para exceções. Todas as outras exceções herdam desta.

aula `psutil.NoSuchProcess` (*pid* , *name = None* , *msg = None*)

Lançada pelos `Process` métodos da classe quando nenhum processo com o *PID* fornecido é encontrado na lista de processos atual ou quando um processo não existe mais. `name` é o nome que o processo tinha antes de desaparecer e é definido apenas se `process.is_name` `Process.name()` foi chamado anteriormente.

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Isso pode ser levantado por `Process` métodos de classe ao consultar um processo zumbi no UNIX (o Windows não possui processos zumbis). Os atributos `*name*` e `*ppid*` estão disponíveis se os métodos `get` `Process.name()` ou `Process.ppid()` `get` foram chamados antes que o processo se tornasse um zumbi.

> **Observação:** esta é uma subclasse de `NoSuchProcess` então se você não estiver interessado em recuperar zumbis (por exemplo, ao usar `process_iter()`) você pode ignorar esta exceção e apenas capturar `NoSuchProcess` .

Adicionado na versão 3.0.0.

aula `psutil.AccessDenied` (*pid = None , name = None , msg = None*)

Gerado por `Process` métodos de classe quando a permissão para executar uma ação é negada devido a privilégios insuficientes. O atributo `*name*` está disponível se `Process.name()` o método foi chamado anteriormente.

aula `psutil.TimeoutExpired` (*segundos , pid = None , name = None , msg = None*)

Lançado pelo `Process.wait()` método se o tempo limite expirar e o processo ainda estiver ativo. O atributo `*name*` estará disponível se `Process.name()` o método foi chamado anteriormente.

Classe Processo

aula `psutil.Process` (*pid = None*)

Representa um processo do sistema operacional com o *PID* fornecido . Se *o PID* for omitido, *o PID* do processo atual (os.getpid) será usado. Uma exceção será lançada `NoSuchProcess` se *o PID* não existir. No Linux, *o PID* também pode se referir a um ID de thread (o campo *id* retornado pelo `threads()` método). Ao acessar métodos desta classe, esteja sempre preparado para capturar `NoSuchProcess` exceções `AccessDenied` . A função interna `hash` pode ser usada em instâncias desta classe para identificar um processo univocamente ao longo do tempo (o hash é determinado pela combinação do PID do processo com a hora de criação). Como tal, ela também pode ser usada com `set` .

> **Observação:** Para obter mais de uma informação sobre o processo simultaneamente de forma eficiente, certifique-se de usar um `oneshot()` gerenciador de contexto ou `as_dict()` um método utilitário.

> **Observação:** A forma como esta classe é vinculada a um processo é através do seu **PID** . Isso significa que, se o processo terminar e o sistema operacional reutilizar o seu PID, você poderá inadvertidamente acabar consultando outro processo. Para evitar esse problema, você pode usar ` first` . Os únicos métodos que verificam `is_running()` preventivamente se o PID foi reutilizado (através do PID + tempo de criação) são : ` `nice()` ( set `ionice()` ) ` , ... `cpu_affinity()` `rlimit()` `children()` `ppid()` `parent()` `parents()` `suspend()` `resume()` `send_signal()` `terminate()` `kill()`

`oneshot` ( )

Gerenciador de contexto utilitário que acelera consideravelmente a recuperação de informações de múltiplos processos simultaneamente. Internamente, diferentes informações de processos (por exemplo `name()` , ` process.ini` `ppid()` , `uids()` ` `create_time()` process.ini`, `process.ini`, etc.) podem ser obtidas usando a mesma rotina, mas apenas um valor é retornado e os demais são descartados. Ao usar este gerenciador de contexto, a rotina interna é executada uma única vez (no exemplo abaixo, em `process.ini` `name()`), o valor de interesse é retornado e os demais são armazenados em cache. As chamadas subsequentes que compartilham a mesma rotina interna retornarão o valor em cache. O cache é limpo ao sair do bloco do gerenciador de contexto. Recomenda-se usar este gerenciador sempre que você precisar recuperar mais de uma informação sobre o processo. Se tiver sorte, você obterá um ganho de velocidade significativo. Exemplo:

```
>>> import psutil
>>> p = psutil.Process()
>>> with p.oneshot():
...     p.name()  # execute internal routine once collecting multiple info
...     p.cpu_times()  # return cached value
...     p.cpu_percent()  # return cached value
...     p.create_time()  # return cached value
...     p.ppid()  # return cached value
...     p.status()  # return cached value
...
>>>
```

Segue uma lista de métodos que podem se beneficiar do aumento de velocidade, dependendo da plataforma que você estiver usando. Na tabela abaixo, as linhas horizontais vazias indicam quais métodos de processo podem ser agrupados internamente de forma eficiente. A última coluna (aceleração) mostra uma estimativa do aumento de velocidade que você pode obter ao chamar todos os métodos juntos (melhor cenário).

Linux	Windows	macOS	BSD	SunOS
`cpu_num()`	`cpu_percent()`	`cpu_percent()`	`cpu_num()`	`name()`
`cpu_percent()`	`cpu_times()`	`cpu_times()`	`cpu_percent()`	`cmdline()`
`cpu_times()`	`io_counters()`	`memory_info()`	`cpu_times()`	`create_time()`
`create_time()`	`memory_info()`	`memory_percent()`	`create_time()`	
`name()`	`memory_maps()`	`num_ctx_switches()`	`gids()`	`memory_info()`
`ppid()`	`num_ctx_switches()`	`num_threads()`	`io_counters()`	`memory_percent()`

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Linux	Windows	macOS	BSD	SunOS
	`username()`	`gids()`	`memory_percent()`	`status()`
`gids()`		`name()`	`num_ctx_switches()`	`terminal()`
`num_ctx_switches()`	`exe()`	`ppid()`	`ppid()`	
`num_threads()`	`name()`	`status()`	`status()`	`gids()`
`uids()`		`terminal()`	`terminal()`	`uids()`
`username()`		`uids()`	`uids()`	`username()`
		`username()`	`username()`	
`memory_full_info()`				
`memory_maps()`				
Aumento de velocidade: +2,6x	*Aumento de velocidade: +1,8x / +6,5x*	*Aumento de velocidade: +1,9x*	*Aumento de velocidade: +2,0x*	*Aumento de velocidade*

Adicionado na versão 5.0.0.

pid

O PID do processo. Este é o único atributo (somente leitura) da classe.

ppid ( )

O PID do processo pai. No Windows, o valor de retorno é armazenado em cache após a primeira chamada. Isso não ocorre em sistemas POSIX, pois o PID pode mudar se o processo se tornar um zumbi. Veja também os métodos ` ppid` `parent()` e `pid` . `parents()`

nome ( )

O nome do processo. No Windows, o valor de retorno é armazenado em cache após a primeira chamada. Isso não ocorre em sistemas POSIX, pois o nome do processo pode mudar. Veja também como encontrar um processo pelo nome .

exe ( )

O executável do processo como um caminho absoluto. Em alguns sistemas, se o executável não puder ser determinado por algum motivo interno (por exemplo, se o processo do sistema ou o caminho não existir mais), este valor pode ser uma string vazia. O valor de retorno é armazenado em cache após a primeira chamada.

```
>>> import psutil
>>> psutil.Process().exe()
'/usr/bin/python3'
```

linha de comando ( )

A linha de comando com a qual este processo foi chamado é uma lista de strings. O valor de retorno não é armazenado em cache porque a linha de comando de um processo pode mudar.

```
>>> import psutil
>>> psutil.Process().cmdline()
['python', 'manage.py', 'runserver']
```

ambiente ( )

As variáveis de ambiente do processo são apresentadas como um dicionário. Observação: isso pode não refletir alterações feitas após o início do processo.

```
>>> import psutil
>>> psutil.Process().environ()
{'LC_NUMERIC': 'it_IT.UTF-8', 'QT_QPA_PLATFORMTHEME': 'appmenu-qt5', 'IM_CONFIG_PHASE': '1', 'XDG_GREETER_DATA_DIR': '/var/lib/lightdm-data/giampaolo',
'XDG_CURRENT_DESKTOP': 'Unity', 'UPSTART_EVENTS': 'started starting', 'GNOME_KEYRING_PID': '', 'XDG_VTNR': '7', 'QT_IM_MODULE': 'ibus', 'LOGNAME':
'giampaolo', 'USER': 'giampaolo', 'PATH':
'/home/giampaolo/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/home/giampaolo/svn/sysconf/bin',
'LC_PAPER': 'it_IT.UTF-8', 'GNOME_KEYRING_CONTROL': '', 'GTK_IM_MODULE': 'ibus', 'DISPLAY': ':0', 'LANG': 'en_US.UTF-8', 'LESS_TERMCAP_se': '\x1b[0m',
'TERM': 'xterm-256color', 'SHELL': '/bin/bash', 'XDG_SESSION_PATH': '/org/freedesktop/DisplayManager/Session0', 'XAUTHORITY': '/home/giampaolo/.Xauthority',
'LANGUAGE': 'en_US', 'COMPIZ_CONFIG_PROFILE': 'ubuntu', 'LC_MONETARY': 'it_IT.UTF-8', 'QT_LINUX_ACCESSIBILITY_ALWAYS_ON': '1', 'LESS_TERMCAP_me': '\x1b[0m',
'LESS_TERMCAP_md': '\x1b[01;38;5;74m', 'LESS_TERMCAP_mb': '\x1b[01;31m', 'HISTSIZE': '100000', 'UPSTART_INSTANCE': '', 'CLUTTER_IM_MODULE': 'xim',
'WINDOWID': '58786407', 'EDITOR': 'vim', 'SESSIONTYPE': 'gnome-session', 'XMODIFIERS': '@im=ibus', 'GPG_AGENT_INFO': '/home/giampaolo/.gnupg/S.gpg-
agent:0:1', 'HOME': '/home/giampaolo', 'HISTFILESIZE': '100000', 'QT4_IM_MODULE': 'xim', 'GTK2_MODULES': 'overlay-scrollbar', 'XDG_SESSION_DESKTOP':
'ubuntu', 'SHLVL': '1', 'XDG_RUNTIME_DIR': '/run/user/1000', 'INSTANCE': 'Unity', 'LC_ADDRESS': 'it_IT.UTF-8', 'SSH_AUTH_SOCK': '/run/user/1000/keyring/ssh',
'VTE_VERSION': '4205', 'GDMSESSION': 'ubuntu', 'MANDATORY_PATH': '/usr/share/gconf/ubuntu.mandatory.path', 'VISUAL': 'vim', 'DESKTOP_SESSION': 'ubuntu',
'QT_ACCESSIBILITY': '1', 'XDG_SEAT_PATH': '/org/freedesktop/DisplayManager/Seat0', 'LESSCLOSE': '/usr/bin/lesspipe %s %s', 'LESSOPEN': '| /usr/bin/lesspipe
%s', 'XDG_SESSION_ID': 'c2', 'DBUS_SESSION_BUS_ADDRESS': 'unix:abstract=/tmp/dbus-9GAJpvnt8r', '_': '/usr/bin/python', 'DEFAULTS_PATH':
'/usr/share/gconf/ubuntu.default.path', 'LC_IDENTIFICATION': 'it_IT.UTF-8', 'LESS_TERMCAP_ue': '\x1b[0m', 'UPSTART_SESSION':
'unix:abstract=/com/ubuntu/upstart-session/1000/1294', 'XDG_CONFIG_DIRS': '/etc/xdg/xdg-ubuntu:/usr/share/upstart/xdg:/etc/xdg', 'GTK_MODULES': 'gail:atk-
bridge:unity-gtk-module', 'XDG_SESSION_TYPE': 'x11', 'PYTHONSTARTUP': '/home/giampaolo/.pythonstart', 'LC_NAME': 'it_IT.UTF-8', 'OLDPWD':
'/home/giampaolo/svn/curio_giampaolo/tests', 'GDM_LANG': 'en_US', 'LC_TELEPHONE': 'it_IT.UTF-8', 'HISTCONTROL': 'ignoredups:erasedups', 'LC_MEASUREMENT':
'it_IT.UTF-8', 'PWD': '/home/giampaolo/svn/curio_giampaolo', 'JOB': 'gnome-session', 'LESS_TERMCAP_us': '\x1b[04;38;5;146m', 'UPSTART_JOB': 'unity-settings-
daemon', 'LC_TIME': 'it_IT.UTF-8', 'LESS_TERMCAP_so': '\x1b[38;5;246m', 'PAGER': 'less', 'XDG_DATA_DIRS':
'/usr/share/ubuntu:/usr/share/gnome:/usr/local/share/:/usr/share/:/var/lib/snapd/desktop', 'XDG_SEAT': 'seat0'}
```

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Adicionado na versão 4.0.0.

Alterado na versão 5.3.0: adicionado suporte ao SunOS

Alterado na versão 5.6.3: adicionado suporte para AIX

Alterado na versão 5.7.3: adicionado suporte a BSD

criar_tempo ( )

O tempo de criação do processo é um número de ponto flutuante expresso em segundos desde a época (segundos desde 1º de janeiro de 1970, à meia-noite UTC). O valor de retorno, que é armazenado em cache após a primeira chamada, é baseado no relógio do sistema, o que significa que pode ser afetado por alterações como ajustes manuais ou sincronização de tempo (por exemplo, NTP).

```
>>> import psutil, datetime
>>> p = psutil.Process()
>>> p.create_time()
1307289803.47
>>> datetime.datetime.fromtimestamp(p.create_time()).strftime("%Y-%m-%d %H:%M:%S")
'2011-03-05 18:03:52'
```

as_dict (*attrs = None , ad_value = None*)

Método utilitário que recupera informações de múltiplos processos como um *dicionário* . Se ` *attrs*` for especificado , deve ser uma lista de strings refletindo os nomes dos atributos disponíveis da classe . `Process` Aqui está *uma* lista de possíveis valores de string : `'cmdline'` ` <string> ` ,

... `'net_connections'` `'cpu_affinity'` `'cpu_num'` `'cpu_percent'` `'cpu_times'` `'create_time'` `'cwd'` `'environ'` `'exe'` `'gids'` `'io_counters'` `'ionice'` `'memory_full_in`

```
>>> import psutil
>>> p = psutil.Process()
>>> p.as_dict(attrs=['pid', 'name', 'username'])
{'username': 'giampaolo', 'pid': 12366, 'name': 'python'}
>>>
>>> # get a list of valid attrs names
>>> list(psutil.Process().as_dict().keys())
['cmdline', 'connections', 'cpu_affinity', 'cpu_num', 'cpu_percent', 'cpu_times', 'create_time', 'cwd', 'environ', 'exe', 'gids', 'io_counters', 'ionice',
'memory_full_info', 'memory_info', 'memory_maps', 'memory_percent', 'name', 'net_connections', 'nice', 'num_ctx_switches', 'num_fds', 'num_threads',
'open_files', 'pid', 'ppid', 'status', 'terminal', 'threads', 'uids', 'username']
```

Alterado na versão 3.0.0: ad_value também é usado quando ocorre uma `ZombieProcess` exceção, não apenas `AccessDenied`

Alterado na versão 4.5.0: `as_dict()` consideravelmente mais rápido graças ao `oneshot()` gerenciador de contexto.

pai ( )

Método utilitário que retorna o processo pai como um `Process` objeto, verificando preventivamente se o PID foi reutilizado. Se nenhum PID pai for conhecido, retorna `None`. Veja também os métodos `ppid()` e `parents()`.

pais ( )

Método utilitário que retorna os pais deste processo como uma lista de `Process` instâncias. Se nenhum pai for conhecido, retorna uma lista vazia. Veja também os métodos `ppid()` `and` `parent()`.

Adicionado na versão 5.6.0.

status ( )

O estado atual do processo como uma string. A string retornada é uma das constantes psutil.STATUS_* .

cwd ( )

O diretório de trabalho atual do processo é retornado como um caminho absoluto. Se o diretório de trabalho atual não puder ser determinado por algum motivo interno (por exemplo, um processo do sistema ou um diretório que não existe mais), ele poderá retornar uma string vazia.

Alterado na versão 5.6.4: adicionado suporte para NetBSD.

nome de usuário ( )

O nome do usuário proprietário do processo. No UNIX, isso é calculado usando o UID real do processo.

uids ( )

Os IDs de usuário reais, efetivos e salvos deste processo são apresentados como uma tupla nomeada. Isso é semelhante a os.getresuid , mas pode ser usado para qualquer PID de processo.

Disponibilidade: UNIX

gids ( )

Os IDs de grupo reais, efetivos e salvos deste processo são apresentados como uma tupla nomeada. Isso é semelhante a os.getresgid , mas pode ser usado para qualquer PID de processo.

Disponibilidade: UNIX

terminal ( )

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Disponibilidade: UNIX

legal (*valor = Nenhum*)

Obtenha ou defina a prioridade (niceness) de um processo. No UNIX, esse valor é um número que geralmente varia de 0 `-20` a 1. `20` Quanto maior o valor de prioridade (niceness), menor a prioridade do processo.

```
>>> import psutil
>>> p = psutil.Process()
>>> p.nice(10)  # set
>>> p.nice()  # get
10
>>>
```

A partir do Python 3.3, essa funcionalidade também está disponível como `os.getpriority` e `os.setpriority` (consulte BPO-10784). No Windows, isso é implementado por meio das APIs `GetPriorityClass` e `SetPriorityClass` do Windows, e *o valor* é uma das `psutil.*_PRIORITY_CLASS` constantes, conforme a documentação da MSDN. Exemplo que aumenta a prioridade de um processo no Windows:

```
>>> p.nice(psutil.HIGH_PRIORITY_CLASS)
```

ionice (*ioclass = Nenhum , valor = Nenhum*)

Obtém ou define a prioridade de E/S do processo. Se nenhum argumento for fornecido, funciona como uma função `get`, retornando uma tupla no Linux e um inteiro `*ioclass*` no Windows. Se `*ioclass*` for fornecido, funciona como uma função `set`. Nesse caso, um *valor* adicional pode ser especificado apenas no Linux para aumentar ou diminuir ainda mais a prioridade de E/S. Aqui estão os possíveis valores de `*ioclass*` , que variam de acordo com a plataforma . `(ioclass, value)`

Linux (consulte o manual do ioprio_get):

- `IOPRIO_CLASS_RT` : (alta) o processo tem acesso prioritário ao disco sempre. Use com cuidado, pois pode sobrecarregar todo o sistema. *Níveis* de prioridade adicionais podem ser especificados, variando de `0` (mais alta) a `7` (mais baixa).
- `IOPRIO_CLASS_BE` : (normal) o padrão para qualquer processo que não tenha definido uma prioridade de E/S específica. *Níveis* de prioridade adicionais variam de `0` (mais alto) a `7` (mais baixo).
- `IOPRIO_CLASS_IDLE` : (baixo) obter tempo de E/S quando ninguém mais precisar do disco. Nenhum *valor* adicional é aceito.
- `IOPRIO_CLASS_NONE` : retornado quando nenhuma prioridade foi definida anteriormente.

Windows:

- `IOPRIO_HIGH` : prioridade máxima.
- `IOPRIO_NORMAL` : prioridade padrão.
- `IOPRIO_LOW` : baixa prioridade.
- `IOPRIO_VERYLOW` : prioridade mais baixa.

Aqui está um exemplo de como definir a prioridade de E/S mais alta, dependendo da plataforma que você estiver usando:

```
>>> import psutil
>>> p = psutil.Process()
>>> if psutil.LINUX:
...     p.ionice(psutil.IOPRIO_CLASS_RT, value=7)
... else:
...     p.ionice(psutil.IOPRIO_HIGH)
...
>>> p.ionice()  # get
pionice(ioclass=<IOPriority.IOPRIO_CLASS_RT: 1>, value=7)
```

Disponibilidade: Linux, Windows Vista+

Alterado na versão 5.6.2: O Windows aceita novas `IOPRIO_*` constantes, incluindo novas `IOPRIO_HIGH` .

rlimit (*recurso , limites = Nenhum*)

Obtém ou define os limites de recursos de um processo (consulte o manual de prlimit). `*resource*` é uma das constantes `psutil.RLIMIT_*` . `*limits*` é uma tupla. Isso é semelhante a `resource.getrlimit` e `resource.setrlimit` , mas pode ser usado para qualquer PID de processo, não apenas `os.getpid` . Para `get`, o valor de retorno é uma tupla. Cada valor pode ser um inteiro ou uma string . Exemplo: `(soft, hard)` `(soft, hard)` `psutil.RLIMIT_*`

```
>>> import psutil
>>> p = psutil.Process()
>>> p.rlimit(psutil.RLIMIT_NOFILE, (128, 128))   # process can open max 128 file descriptors
>>> p.rlimit(psutil.RLIMIT_FSIZE, (1024, 1024))  # can create files no bigger than 1024 bytes
>>> p.rlimit(psutil.RLIMIT_FSIZE)                # get
(1024, 1024)
>>>
```

Veja também o script procinfo.py .

Disponibilidade: Linux, FreeBSD

Alterado na versão 5.7.3: adicionado suporte ao FreeBSD

contadores_io ( )

Retorna as estatísticas de E/S do processo como uma tupla nomeada. Para Linux, você pode consultar a documentação do siste

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

- **read_count** : o número de operações de leitura realizadas (acumulado). Supõe-se que isso conte o número de chamadas de sistema relacionadas à leitura, como read_read `read()` e read_read_write `pread()` no UNIX.
- **write_count** : o número de operações de escrita realizadas (acumulado). Supõe-se que isso conte o número de chamadas de sistema relacionadas à escrita, como write `write()` e write `pwrite()` no UNIX.
- **read_bytes** : o número de bytes lidos (cumulativo). Sempre presente `-1` no BSD.
- **write_bytes** : o número de bytes gravados (acumulado). Sempre presente `-1` no BSD.

Específico para Linux:

- **read_chars** *(Linux)* : a quantidade de bytes que este processo passou para `read()` chamadas `pread()` de sistema (cumulativa). Diferentemente de *read_bytes*, não leva em consideração se houve ou não E/S física real no disco.
- **write_chars** *(Linux)* : a quantidade de bytes que este processo passou para chamadas `write()` de `pwrite()` sistema (cumulativa). Diferentemente de *write_bytes*, não leva em consideração se houve ou não E/S física real no disco.

Específico para Windows:

- **other_count** *(Windows)* : o número de operações de E/S realizadas que não sejam operações de leitura e gravação.
- **other_bytes** *(Windows)* : o número de bytes transferidos durante operações que não sejam de leitura ou gravação.

```
>>> import psutil
>>> p = psutil.Process()
>>> p.io_counters()
pio(read_count=454556, write_count=3456, read_bytes=110592, write_bytes=0, read_chars=769931, write_chars=203)
```

Disponibilidade: Linux, BSD, Windows, AIX

Alterações na versão 5.2.0: adicionadas *as funções read_chars* e *write_chars* no Linux; adicionadas as funções *other_count* e *other_bytes* no Windows.

num_ctx_switches ( )

Número de mudanças de contexto voluntárias e involuntárias realizadas por esse processo (acumulativo).

Alterado na versão 5.4.1: adicionado suporte para AIX

num_fds ( )

Número de descritores de arquivo atualmente abertos por este processo (não cumulativo).

Disponibilidade: UNIX

num_handles ( )

Número de identificadores atualmente utilizados por este processo (não cumulativo).

Disponibilidade: Windows

num_threads ( )

Número de threads atualmente em uso por este processo (não cumulativo).

threads ( )

Retorna as threads abertas pelo processo como uma lista de tuplas nomeadas. No OpenBSD, este método requer privilégios de root.

- **`id`** : o ID da thread nativa atribuído pelo kernel. Se `pid` se referir ao processo atual, corresponde ao atributo `native_id` da classe `threading.Thread` e pode ser usado para referenciar threads Python individuais em execução dentro do seu próprio aplicativo Python.
- **user_time** : tempo gasto no modo de usuário.
- **system_time** : tempo gasto no modo kernel.

cpu_times ( )

Retorna uma tupla nomeada representando os tempos de processo acumulados, em segundos (veja a explicação). Isso é semelhante a os.times , mas pode ser usado para qualquer PID de processo.

- **usuário** : tempo gasto no modo de usuário.
- **Sistema** : tempo gasto no modo kernel.
- **children_user** : tempo de usuário de todos os processos filhos (sempre presente `0` no Windows e macOS).
- **children_system** : hora do sistema de todos os processos filhos (sempre presente `0` no Windows e macOS).
- **iowait** : (Linux) tempo gasto aguardando a conclusão de operações de E/S bloqueantes. Esse valor é excluído da contagem de tempo *do usuário* e *do sistema* (porque a CPU não está executando nenhuma tarefa).

```
>>> import psutil
>>> p = psutil.Process()
>>> p.cpu_times()
pcputimes(user=0.03, system=0.67, children_user=0.0, children_system=0.0, iowait=0.08)
>>> sum(p.cpu_times()[:2])  # cumulative, excluding children and iowait
0.70
```

Alterado na versão 4.1.0: retorna dois campos extras: *children_user* e *children_system* .

Alterado na versão 5.6.4: adicionada *a função iowait* no Linux.

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Retorna um valor float representando a utilização da CPU do processo em porcentagem, que também pode ser um número inteiro caso o processo esteja executando múltiplas threads em CPUs diferentes. Quando *o intervalo* for maior que o especificado, compara o tempo de processamento do processo com o tempo de CPU do sistema decorrido antes e depois do intervalo (bloqueio). Quando o intervalo for menor ou igual ao especificado, compara o tempo de processamento do processo com o tempo de CPU do sistema decorrido desde a última chamada, retornando imediatamente. Isso significa que, na primeira vez que a função for chamada, ela retornará um valor sem significado, que deve ser ignorado. Nesse caso, recomenda-se, para maior precisão, que a função seja chamada uma segunda vez com um intervalo de pelo menos 10 segundos entre as chamadas. Exemplo: `>`

`100.0` `0.0` `0.0` `None` `0.0` `0.1`

```
>>> import psutil
>>> p = psutil.Process()
>>> # blocking
>>> p.cpu_percent(interval=1)
2.0
>>> # non-blocking (percentage since last call)
>>> p.cpu_percent(interval=None)
2.9
```

Observação: O valor retornado pode ser > 100,0 no caso de um processo que execute várias threads em diferentes núcleos de CPU.

Observação: O valor retornado *não* é explicitamente dividido igualmente entre todas as CPUs disponíveis (diferentemente de `lscp` `psutil.cpu_percent()`). Isso significa que um processo em loop ocupado, executado em um sistema com 2 CPUs lógicas, será relatado com 100% de utilização da CPU em vez de 50%. Isso foi feito para ser consistente com `top` o utilitário `lscp` do UNIX e também para facilitar a identificação de processos que consomem muitos recursos da CPU, independentemente do número de CPUs. Deve-se notar que `taskmgr.exe` no Windows o `lscp` não se comporta dessa maneira (ele relataria 50% de uso). Para emular `taskmgr.exe` o comportamento do Windows, você pode fazer: `lscp`. `p.cpu_percent() / psutil.cpu_count()`

Aviso: Na primeira vez que este método for chamado com `interval = 0.0 or`, `None` ele retornará um `0.0` valor sem significado que você deve ignorar.

`afinidade_cpu` (*cpus = None*)

Obtém ou define a afinidade de CPU atual de um processo . A afinidade de CPU consiste em instruir o sistema operacional a executar um processo apenas em um conjunto limitado de CPUs (no terminal Linux, `taskset` o comando `cpu` é normalmente usado). Se nenhum argumento for passado, retorna a afinidade de CPU atual como uma lista de inteiros. Se um argumento for passado, deve ser uma lista de inteiros especificando a nova afinidade de CPU. Se uma lista vazia for passada, todas as CPUs elegíveis serão consideradas (e definidas). Em alguns sistemas, como o Linux, isso pode não significar necessariamente todas as CPUs lógicas disponíveis (como em `cpu` `list(range(psutil.cpu_count()))`).

```
>>> import psutil
>>> psutil.cpu_count()
4
>>> p = psutil.Process()
>>> # get
>>> p.cpu_affinity()
[0, 1, 2, 3]
>>> # set; from now on, process will run on CPU #0 and #1 only
>>> p.cpu_affinity([0, 1])
>>> p.cpu_affinity()
[0, 1]
>>> # reset affinity against all eligible CPUs
>>> p.cpu_affinity([])
```

Disponibilidade: Linux, Windows, FreeBSD

Alterado na versão 2.2.0: adicionado suporte para FreeBSD.

Alterado na versão 5.1.0: uma lista vazia pode ser passada para definir a afinidade com todas as CPUs elegíveis.

`cpu_num` ( )

Retorna em qual CPU este processo está sendo executado no momento. O número retornado deve ser `<=` `psutil.cpu_count()`. No FreeBSD, certos processos do kernel podem retornar `-1`. Ele pode ser usado em conjunto com `psutil.cpu_percent(percpu=True)` para observar a carga de trabalho do sistema distribuída entre várias CPUs, como mostrado no script de exemplo cpu_distribution.py .

Disponibilidade: Linux, FreeBSD, SunOS

Adicionado na versão 5.1.0.

`informações_de_memória` ( )

Retorna uma tupla nomeada com campos variáveis dependendo da plataforma, representando informações de memória sobre o processo. Os campos "portáteis" disponíveis em todas as plataformas são `rss` e `vms` . Todos os números são expressos em bytes.

Linux	macOS	BSD	Solaris	AIX	Windows
rss	rss	rss	rss	rss	rss (alias para `wset`)
máquinas virtuais	máquinas virtuais	máquinas virtuais	máquinas virtuais	máquinas virtuais	vms (alias para `pagefile`)
compartilhado	falhas	texto			num_page_faults
texto	páginas	dados			pico_wset

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Linux	macOS	BSD	Solaris	AIX	Windows
dados					pico_paginado_pool
sujo					pool_paginado
					pico_não_paginado_pool
					pool não paginado
					arquivo de paginação
					arquivo_de_página_pico
					privado

- **rss** : também conhecido como "Resident Set Size" (Tamanho do Conjunto Residente), representa a memória física não trocada que um processo utilizou. No UNIX, corresponde à coluna RES do comando "top". No Windows, é um alias para o campo *wset* e corresponde à coluna "Mem Usage" (Uso de Memória) do taskmgr.exe.
- **vms** : também conhecido como "Tamanho da Memória Virtual", este valor representa a quantidade total de memória virtual utilizada pelo processo. No UNIX, corresponde à coluna VIRT do comando "top". No Windows, é um alias para o campo *do arquivo de paginação* e corresponde à coluna "Uso de Memória" e "Tamanho da VM" do taskmgr.exe.
- **Compartilhada** : *(Linux)* memória que pode ser potencialmente compartilhada com outros processos. Isso corresponde à coluna SHR do comando "top".
- **texto** *(Linux, BSD)* : também conhecido como TRS (conjunto residente de texto), a quantidade de memória dedicada ao código executável. Isso corresponde à coluna CODE do comando "top".
- **dados** *(Linux, BSD)* : também conhecido como DRS (conjunto de dados residentes), a quantidade de memória física dedicada a código que não seja executável. Corresponde à coluna DATA do comando "top".
- **lib** *(Linux)* : a memória usada pelas bibliotecas compartilhadas.
- **dirty** *(Linux)* : o número de páginas sujas.
- **pfaults** *(macOS)* : número de falhas de página.
- **pageins** *(macOS)* : número de paginações reais.

Para obter uma explicação sobre os campos do Windows, consulte a documentação da estrutura PROCESS_MEMORY_COUNTERS_EX . Exemplo no Linux:

```
>>> import psutil
>>> p = psutil.Process()
>>> p.memory_info()
pmem(rss=15491072, vms=84025344, shared=5206016, text=2555904, lib=0, data=9891840, dirty=0)
```

Alterado na versão 4.0.0: vários campos são retornados, não apenas *rss* e *vms* .

memory_full_info ( )

Este método retorna as mesmas informações que o `memory_info()` anterior, além de, em algumas plataformas (Linux, macOS, Windows), fornecer métricas adicionais (USS, PSS e swap). As métricas adicionais oferecem uma representação melhor do consumo "efetivo" de memória do processo (no caso do USS), conforme explicado detalhadamente nesta postagem do blog . Isso é feito passando o endereço completo do processo. Como tal, geralmente requer privilégios de usuário mais elevados do que o anterior `memory_info()` e é consideravelmente mais lento. Em plataformas onde os campos extras não estão implementados, este método simplesmente retorna as mesmas métricas que o anterior `memory_info()` .

- **uss** *(Linux, macOS, Windows)* : também conhecido como "Unique Set Size" (Tamanho de Conjunto Único), é a memória exclusiva de um processo e que seria liberada se o processo fosse encerrado imediatamente.
- **PSS** *(Linux)* : também conhecido como "Tamanho de Conjunto Proporcional", é a quantidade de memória compartilhada com outros processos, contabilizada de forma que essa quantidade seja dividida igualmente entre os processos que a compartilham. Ou seja, se um processo tiver 10 MB dedicados exclusivamente a ele e 10 MB compartilhados com outro processo, seu PSS será de 15 MB.
- **swap** *(Linux)* : quantidade de memória que foi trocada para o disco.

> **Observação:** *O uss* é provavelmente a métrica mais representativa para determinar quanta memória está sendo realmente usada por um processo. Ele representa a quantidade de memória que seria liberada se o processo fosse encerrado neste momento.

Exemplo no Linux:

```
>>> import psutil
>>> p = psutil.Process()
>>> p.memory_full_info()
pfullmem(rss=10199040, vms=52133888, shared=3887104, text=2867200, lib=0, data=5967872, dirty=0, uss=6545408, pss=6872064, swap=0)
>>>
```

Veja também o arquivo procsmem.py para um exemplo de aplicação.

Adicionado na versão 4.0.0.

porcentagem_de_memória (*tipo_memória = 'rss'*)

Compara a memória do processo com a memória física total do sistema e calcula a utilização da memória do processo em porce

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Alterado na versão 4.0.0: adicionado o parâmetro *memtype* .

`mapas_de_memória` (*agrupado = Verdadeiro*)

Retorna as regiões de memória mapeadas do processo como uma lista de tuplas nomeadas cujos campos variam dependendo da plataforma. Este método é útil para obter uma representação detalhada do uso de memória do processo, conforme explicado aqui (o valor mais importante é a memória "privada"). Se o parâmetro `*grouped*` estiver definido, `True` as regiões mapeadas com o mesmo *caminho* são agrupadas e os diferentes campos de memória são somados. Caso *contrário* , cada região mapeada é exibida como uma entidade individual e a tupla nomeada também incluirá o espaço de endereçamento (`*addr*`) e o conjunto de permissões (`*perms*`) `False` da região mapeada . Consulte o arquivo `pmap.py` para um exemplo de aplicação.

Linux	Windows	FreeBSD	Solaris
rss	rss	rss	rss
tamanho		privado	anônimo
pss		contagem_de_referências	trancado
limpeza compartilhada		contagem de sombras	
compartilhado_sujo			
limpeza privada			
privado_sujo			
referenciado			
anônimo			
trocar			

```
>>> import psutil
>>> p = psutil.Process()
>>> p.memory_maps()
[pmmap_grouped(path='/lib/x8664-linux-gnu/libutil-2.15.so', rss=32768, size=2125824, pss=32768, shared_clean=0, shared_dirty=0, private_clean=20480,
private_dirty=12288, referenced=32768, anonymous=12288, swap=0),
 pmmap_grouped(path='/lib/x8664-linux-gnu/libc-2.15.so', rss=3821568, size=3842048, pss=3821568, shared_clean=0, shared_dirty=0, private_clean=0,
private_dirty=3821568, referenced=3575808, anonymous=3821568, swap=0),
 ...]
```

Disponibilidade: Linux, Windows, FreeBSD, SunOS

Alterado na versão 5.6.0: suporte ao macOS removido por ser inerentemente problemático (ver problema nº 1291).

`filhos` (*recursivo = Falso*)

Retorna os filhos deste processo como uma lista de `Process` instâncias. Se `recursive` for `*True*` , retorna todos os descendentes do processo pai. Exemplo de pseudocódigo assumindo que `*A == this process*` :

```
A ─┐
   ├─ B (child) ─┐
   │             └─ X (grandchild) ─┐
   │                                └─ Y (great grandchild)
   ├─ C (child)
   └─ D (child)

>>> p.children()
B, C, D
>>> p.children(recursive=True)
B, X, Y, C, D
```

Note que, no exemplo acima, se o processo X desaparecer, o processo Y também não será retornado, pois a referência ao processo A é perdida. Esse conceito é bem resumido por este teste unitário . Veja também como encerrar uma árvore de processos e finalizar meus processos filhos .

`abrir_arquivos` ( )

Retorna os arquivos regulares abertos pelo processo como uma lista de tuplas nomeadas, incluindo os seguintes campos:

- **caminho** : o nome absoluto do arquivo.
- **fd** : o número do descritor de arquivo; no Windows, este é sempre `-1` .

Apenas para Linux:

- **posição** (*Linux*): a posição (deslocamento) do arquivo.
- **modo** (*Linux*): uma string que indica como o arquivo foi aberto, similar ao argumento interno do comando open `mode` . Os valores possíveis são `'r'` , `'w'` , `'a'` , `'r+'` e `'a+'` . Não há distinção entre arquivos abertos em modo binário ou texto (`"b"` ou `"t"`).
- **flags** (*Linux*): os sinalizadores que foram passados para a chamada C os.open subjacente quando o arquivo foi aberto (por exemplo, os.O_RDONLY , os.O_TRUNC , etc).

```
>>> import psutil
```

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Aviso: No Windows, esse método não é confiável devido a algumas limitações da API subjacente do Windows, que pode travar ao recuperar determinados identificadores de arquivo. Para contornar isso, o psutil cria uma thread para determinar o nome do identificador de arquivo e a encerra se ela não responder após 100 ms. Isso implica que esse método no Windows não garante a enumeração de todos os identificadores de arquivo regulares (veja a issue 597). Ferramentas como o ProcessHacker têm a mesma limitação.

Aviso: No BSD, este método pode retornar arquivos com um caminho nulo ("") devido a um bug do kernel, portanto não é confiável (veja a issue 595).

Alterado na versão 3.1.0: não trava mais no Windows.

Alterações na versão 4.1.0: novos campos *de posição* , *modo* e *sinalizadores* no Linux.

conexões_de_rede (*tipo = 'inet'*)

Retorna as conexões de socket abertas pelo processo como uma lista de tuplas nomeadas. Para obter as conexões de todo o sistema, use `psutil.net_connections()`. Cada tupla nomeada fornece 6 atributos:

- **fd** : o descritor de arquivo do socket. Se a conexão se referir ao processo atual, este pode ser passado para socket.fromfd para obter um objeto de socket utilizável. No Windows, FreeBSD e SunOS, este valor é sempre definido como `-1`.
- **família** : a família de endereços, seja AF_INET , AF_INET6 ou AF_UNIX .
- **tipo** : o tipo de endereço, seja SOCK_STREAM , SOCK_DGRAM ou SOCK_SEQPACKET .
- **laddr** : o endereço local como uma tupla nomeada ou um valor no caso de sockets AF_UNIX. Para sockets UNIX, consulte as notas abaixo. `(ip, port)` `path`
- **raddr** : o endereço remoto como uma tupla nomeada ou um valor absoluto no caso de sockets UNIX. Quando o endpoint remoto não estiver conectado, você receberá uma tupla vazia (AF_INET*) ou (AF_UNIX). Para sockets UNIX, consulte as notas abaixo. `(ip, port)` `path` `""`
- **status** : representa o status de uma conexão TCP. O valor de retorno é uma das `psutil.CONN_*` constantes. Para sockets UDP e UNIX, este valor será sempre `psutil.CONN_NONE`.

O parâmetro `*kind*` é uma string que filtra as conexões que atendem aos seguintes critérios:

Valor de tipo	Conexões usando
`"inet"`	IPv4 e IPv6
`"inet4"`	IPv4
`"inet6"`	IPv6
`"tcp"`	TCP
`"tcp4"`	TCP sobre IPv4
`"tcp6"`	TCP sobre IPv6
`"udp"`	UDP
`"udp4"`	UDP sobre IPv4
`"udp6"`	UDP sobre IPv6
`"unix"`	Socket UNIX (protocolos UDP e TCP)
`"all"`	a soma de todas as famílias e protocolos possíveis

Exemplo:

```
>>> import psutil
>>> p = psutil.Process(1694)
>>> p.name()
'firefox'
>>> p.net_connections()
[pconn(fd=115, family=<AddressFamily.AF_INET: 2>, type=<SocketType.SOCK_STREAM: 1>, laddr=addr(ip='10.0.0.1', port=48776), raddr=addr(ip='93.186.135.91',
port=80), status='ESTABLISHED'),
 pconn(fd=117, family=<AddressFamily.AF_INET: 2>, type=<SocketType.SOCK_STREAM: 1>, laddr=addr(ip='10.0.0.1', port=43761), raddr=addr(ip='72.14.234.100',
port=80), status='CLOSING'),
 pconn(fd=119, family=<AddressFamily.AF_INET: 2>, type=<SocketType.SOCK_STREAM: 1>, laddr=addr(ip='10.0.0.1', port=60759), raddr=addr(ip='72.14.234.104',
port=80), status='ESTABLISHED'),
 pconn(fd=123, family=<AddressFamily.AF_INET: 2>, type=<SocketType.SOCK_STREAM: 1>, laddr=addr(ip='10.0.0.1', port=51314), raddr=addr(ip='72.14.234.83',
port=443), status='SYN_SENT')]
```

Aviso: No Linux, a obtenção de conexões para determinados processos requer privilégios de root. Se o psutil não for executado como root, essas conexões serão ignoradas silenciosamente, em vez de gerar uma exceção `psutil.AccessDenied`. Isso significa que a lista retornada pode estar incompleta.

Observação: (Solaris) Sockets UNIX não são suportados.

Observação: (Linux, FreeBSD) O campo *raddr* para sockets UNIX é sempre definido como "". Esta é uma limitação do sistema

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Observação: (AIX) `psutil.AccessDenied` é sempre levantado, a menos que esteja sendo executado como root (lsof faz o mesmo).

Alterado na versão 5.3.0: laddr e raddr *agora* são tuplas nomeadas.

Alterado na versão 6.0.0: método renomeado de *connections* para *net_connections* .

conexões ( )

O mesmo que `net_connections()` (obsoleto).

Aviso: Obsoleto na versão 6.0.0; use `net_connections()` em vez disso.

está_em_execução ( )

Retorna se o processo atual está em execução na lista de processos atuais. Isso é confiável mesmo se o processo for encerrado e seu PID for reutilizado por outro processo; portanto, deve ser preferido em relação a usar ` `psutil.pid_exists(p.pid)` . Se o PID tiver sido reutilizado, este método também removerá o processo do `process_iter()` cache interno.

Observação: isso `True` também será retornado se o processo for um zumbi ( ). `p.status() == psutil.STATUS_ZOMBIE`

Alterado na versão 6.0.0: remove automaticamente o processo do `process_iter()` cache interno se o PID tiver sido reutilizado por outro processo.

enviar_sinal (*sinal*)

Envia um sinal para o processo (consulte as constantes do módulo de sinal) verificando preventivamente se o PID foi reutilizado. No UNIX, isso é o mesmo que ` sigterm` . No Windows, apenas os sinais `*SIGTERM*` , `*CTRL_C_EVENT*` e `*CTRL_BREAK_EVENT*` são suportados, e `*SIGTERM*` é tratado como um alias para `sigterm` . Veja também como encerrar uma árvore de processos e encerrar meus processos filhos . `os.kill(pid, sig)` `kill()`

Alterado na versão 3.2.0: foi adicionado suporte para os sinais CTRL_C_EVENT e CTRL_BREAK_EVENT no Windows.

suspender ( )

Suspenda a execução do processo com o sinal *SIGSTOP* , verificando preventivamente se o PID foi reutilizado. No UNIX, isso é equivalente a [comando omitido] . No Windows, isso é feito suspendendo a execução de todas as threads do processo. `os.kill(pid, signal.SIGSTOP)`

retomar ( )

Retome a execução do processo com o sinal *SIGCONT* , verificando preventivamente se o PID foi reutilizado. No UNIX, isso é equivalente a [comando omitido] . No Windows, isso é feito retomando a execução de todas as threads do processo. `os.kill(pid, signal.SIGCONT)`

terminar ( )

Encerre o processo com o sinal *SIGTERM* , verificando preventivamente se o PID foi reutilizado. No UNIX, isso é o mesmo que `kill` . No Windows, é um alias para `kill` . Veja também como matar uma árvore de processos e encerrar meus processos filhos . `os.kill(pid, signal.SIGTERM)` `kill()`

matar ( )

Mate o processo atual usando o sinal *SIGKILL* , verificando preventivamente se o PID foi reutilizado. No UNIX, isso é feito da mesma forma que [comando omitido]. No Windows, isso é feito usando TerminateProcess . Veja também como matar uma árvore de processos e encerrar meus processos filhos . `os.kill(pid, signal.SIGKILL)`

aguarde (*tempo limite = Nenhum*)

Aguarde a finalização do processo com o PID especificado. Os detalhes sobre o valor de retorno diferem entre UNIX e Windows.

Em sistemas UNIX : se o processo terminar normalmente, o valor de retorno é um inteiro positivo >= 0, indicando o código de saída. Se o processo foi terminado por um sinal, retorna o valor negado do sinal que causou o término (por exemplo, `--no-sinal` `-SIGTERM`). Se o PID não for um processo filho de `os.getpid` (processo atual), aguarda-se até que o processo seja finalizado e retorna-se `null` `None` . Se o PID não existir, retorna-se `None` imediatamente.

No Windows : sempre retorne o código de saída, que é um número inteiro positivo retornado por GetExitCodeProcess .

O tempo limite é expresso em segundos. Se especificado e o processo ainda estiver ativo, `TimeoutExpired` uma exceção será lançada. `timeout=0` Pode ser usado em aplicativos não bloqueantes: ele retornará imediatamente ou lançará uma exceção `TimeoutExpired` .

O valor de retorno é armazenado em cache. Para aguardar vários processos, use `psutil.wait_procs()` .

```
>>> import psutil
>>> p = psutil.Process(9891)
>>> p.terminate()
>>> p.wait()
<Negsignal.SIGTERM: -15>
```

Observação: Quando `timeout` isso não ocorre `None` e a plataforma o suporta, um mecanismo eficiente orientado a eventos é usado para aguardar o término do processo:

- Linux >= 5.3 com Python >= 3.9 usa os.pidfd_open + select.poll
- macOS e outras variantes do BSD usam select.kqueue + `KQ_FILTER_PROC` + `KQ_NOTE_EXIT`

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Caso nenhum desses mecanismos esteja disponível, a função recorre a um loop de espera (chamada não bloqueante e breves pausas).

Alterado na versão 3.15: se *o tempo limite* não for definido `None`, use a implementação eficiente orientada a eventos no Linux >= 5.3 e macOS/BSD.

Alterado na versão 5.7.1: o valor de retorno é armazenado em cache (em vez de retornar `None`).

Alterado na versão 5.7.1: em sistemas POSIX, em caso de sinal negativo, retorne-o como uma enumeração legível por humanos .

Alterado na versão 7.2.2: em Linux >= 5.3 + Python >= 3.9 e macOS/BSD, use os.pidfd_open e select.kqueue , respectivamente, em vez da sondagem de loop ocupado menos eficiente.

aula `psutil.Popen` (** args , ** kwargs*)

Semelhante a subprocess.Popen , mas com a adição de fornecer todos `psutil.Process` os métodos em uma única classe. Para os seguintes métodos, comuns a ambas as classes, a implementação de psutil tem precedência: `send_signal()` , `terminate()` , `kill()` . Isso é feito para evitar o encerramento de outro processo caso seu PID tenha sido reutilizado, corrigindo o problema BPO-6973 .

```
>>> import psutil
>>> from subprocess import PIPE
>>>
>>> p = psutil.Popen(["/usr/bin/python", "-c", "print('hello')"], stdout=PIPE)
>>> p.name()
'python'
>>> p.username()
'giampaolo'
>>> p.communicate()
('hello\n', None)
>>> p.wait(timeout=2)
0
>>>
```

Alterado na versão 4.4.0: adicionado suporte ao gerenciador de contexto.

introspecção de C barato

As funções a seguir fornecem acesso direto ao alocador de memória heap nativo da plataforma C (como o da glibc `malloc` no Linux ou `jemalloc` no BSD). São interfaces de baixo nível destinadas a detectar vazamentos de memória em extensões C, que geralmente não são revelados pelas métricas padrão do RSS/VMS. Essas funções não refletem a memória de objetos Python; elas operam exclusivamente em alocações feitas em C por meio de `malloc` `malloc()` , `malloc` `free()` e chamadas relacionadas.

A ideia geral por trás dessas funções é simples: capturar o estado do heap C antes e depois de invocar repetidamente uma função implementada em uma extensão C e comparar os resultados. Se ` `heap_used` x` ou `y` `mmap_used` crescerem de forma constante ao longo das iterações, é provável que o código C esteja retendo memória que deveria liberar. Isso fornece uma maneira, no nível do alocador, de detectar vazamentos nativos que o rastreamento de memória do Python não detecta.

Confira o projeto psleak para ver um exemplo prático de como essas APIs podem ser usadas para detectar vazamentos de memória em extensões C.

`psutil.heap_info` ( )

Retorna estatísticas de baixo nível do heap do alocador C do sistema. No Linux, isso expõe os campos `maininfo` `uordblks` e `data` da biblioteca `mallinof2` `hblkhd` da glibc . Retorna uma tupla nomeada contendo:

- `heap_used` : número total de bytes atualmente alocados via `malloc()` (alocações pequenas).
- `mmap_used` : número total de bytes atualmente alocados via `save` `mmap()` ou via alocações grandes `malloc()` . Sempre definido como 0 no macOS.
- `heap_count` : (Apenas para Windows) número de heaps privados criados via `HeapCreate()` .

```
>>> import psutil
>>> psutil.heap_info()
pheap(heap_used=5177792, mmap_used=819200)
```

Esses campos refletem como as alocações C não liberadas afetam o heap:

Plataforma	Tipo de alocação	Campo afetado
UNIX / glibc	pequeno `malloc()` ≤128 KB sem `free()`	`heap_used`
UNIX / glibc	grande `malloc()` >128KB sem `free()` , ou `mmap()` sem `munmap()`	`mmap_used`
Windows	`HeapAlloc()` sem `HeapFree()`	`heap_used`
Windows	`VirtualAlloc()` sem `VirtualFree()`	`mmap_used`
Windows	`HeapCreate()` sem `HeapDestroy()`	`heap_count`

Adicionado na versão 7.2.0.

Disponibilidade: Linux + glibc (ou seja, não MUSL), Windows, macOS, FreeBSD, NetBSD

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Solicite ao alocador subjacente que libere qualquer memória não utilizada que esteja armazenando no heap (normalmente `malloc()` alocações pequenas).

Na prática, os alocadores modernos raramente se adequam a essa exigência, portanto, esta não é uma ferramenta de redução de memória de propósito geral e não reduzirá significativamente o RSS em programas reais. Seu principal valor reside em **ferramentas de detecção de vazamentos de memória** .

Chamar o `heap_trim()` método `get_allocator()` antes de realizar as medições ajuda a reduzir o ruído do alocador, fornecendo uma linha de base mais limpa para que as alterações `heap_used` provenham do código que você está testando, e não do cache interno ou da fragmentação do alocador. Sua eficácia depende do comportamento do alocador e dos padrões de fragmentação.

Adicionado na versão 7.2.0.

Disponibilidade: Linux + glibc (ou seja, não MUSL), Windows, macOS, FreeBSD, NetBSD

Serviços do Windows

`psutil.win_service_iter ( )`

Retorna um iterador que produz uma `WindowsService` instância de classe para todos os serviços do Windows instalados.

Adicionado na versão 4.2.0.

Disponibilidade: Windows

`psutil.win_service_get ( nome )`

Obtém um serviço do Windows pelo nome, retornando uma `WindowsService` instância. Gera uma exceção `psutil.NoSuchProcess` se nenhum serviço com esse nome existir.

Adicionado na versão 4.2.0.

Disponibilidade: Windows

aula `psutil.Serviço do Windows`

Representa um serviço do Windows com o *nome* fornecido . Esta classe é retornada pelas funções `start` `win_service_iter()` e `win_service_get()` `end` e não deve ser instanciada diretamente.

`nome ( )`

O nome do serviço. Essa string é como um serviço é referenciado e pode ser passada para `win_service_get()` obter uma nova `WindowsService` instância.

`nome de exibição ( )`

O nome de exibição do serviço. O valor é armazenado em cache quando esta classe é instanciada.

`binpath ( )`

O caminho completo para o arquivo binário/exe do serviço, em formato de string, incluindo os argumentos da linha de comando.

`nome de usuário ( )`

O nome do usuário que possui este serviço.

`tipo_inicial ( )`

Uma sequência de caracteres que pode ser *"automática"* , *"manual"* ou *"desativada"* .

`pid ( )`

O PID do processo, se houver, caso contrário, *None* . Este valor pode ser passado para a `Process` classe para controlar o processo do serviço.

`status ( )`

O status do serviço é representado como uma string, que pode ser *"running"* , *"paused"* , *"start_pending"* , *"pause_pending"* , *"continue_pending"* , *"stop_pending"* ou *"stopped"* .

`descrição ( )`

Descrição detalhada do serviço.

`como_dict ( )`

Método utilitário que recupera todas as informações acima em um dicionário.

Adicionado na versão 4.2.0.

Disponibilidade: Windows

Código de exemplo:

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

```
>>> import psutil
>>> list(psutil.win_service_iter())
[<WindowsService(name='AeLookupSvc', display_name='Application Experience') at 38850096>,
 <WindowsService(name='ALG', display_name='Application Layer Gateway Service') at 38850128>,
 <WindowsService(name='APNMCP', display_name='Ask Update Service') at 38850160>,
 <WindowsService(name='AppIDSvc', display_name='Application Identity') at 38850192>,
 ...]
>>> s = psutil.win_service_get('alg')
>>> s.as_dict()
{'binpath': 'C:\\Windows\\System32\\alg.exe',
 'description': 'Provides support for 3rd party protocol plug-ins for Internet Connection Sharing',
 'display_name': 'Application Layer Gateway Service',
 'name': 'alg',
 'pid': None,
 'start_type': 'manual',
 'status': 'stopped',
 'username': 'NT AUTHORITY\\LocalService'}
```

Constantes

Constantes do sistema operacional

psutil. POSIX

psutil. LINUX

psutil. WINDOWS

psutil. MACOS

psutil. FREEBSD

psutil.NETBSD

psutil. OPENBSD

psutil. BSD

psutil. SUNOS

psutil. AIX

`bool` Constantes que definem em qual plataforma você está. Por exemplo, se estiver no Windows, `WINDOWS` a constante será `True`, para todas as outras será `False`.

Adicionado na versão 4.0.0.

Alterado na versão 5.4.0: adicionado suporte para AIX.

psutil. OSX

Apelido para `MACOS`.

> **Aviso:** Obsoleto na versão 5.4.7; use `MACOS` em vez disso.

psutil. PROCFS_PATH

O caminho do sistema de arquivos /proc no Linux, Solaris e AIX (o padrão é `"/proc"`). Você pode querer redefinir essa constante logo após importar o psutil caso seu sistema de arquivos /proc esteja montado em outro local ou se desejar obter informações sobre contêineres Linux como Docker, Heroku ou LXC (veja aqui para mais informações). Deve-se observar que esse truque funciona apenas para APIs que dependem do sistema de arquivos /proc (por exemplo, APIs de memória `Process` e a maioria dos métodos de classe).

Disponibilidade: Linux, Solaris, AIX

Adicionado na versão 3.2.3.

Alterado na versão 3.4.2: também disponível no Solaris.

Alterado na versão 5.4.0: também disponível no AIX.

Constantes de estado do processo

psutil.STATUS_RUNNING

psutil.STATUS_SLEEPING

psutil.STATUS_DISK_SLEEP

psutil.STATUS_PARADO

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

psutil. STATUS_ZOMBIE

psutil. STATUS_DEAD

psutil.STATUS_WAKE_KILL

psutil.STATUS_WAKING

psutil.STATUS_PARKED (*Linux*)

psutil.STATUS_IDLE (*Linux* , *macOS* , *FreeBSD*)

psutil.STATUS_LOCKED (*FreeBSD*)

psutil.STATUS_WAITING (*FreeBSD*)

psutil. STATUS_SUSPENDED (*NetBSD*)

Representa o estado de um processo. Retornado por `psutil.Process.status()`.

Adicionado na versão 3.4.1: `STATUS_SUSPENDED` (NetBSD)

Adicionado na versão 5.4.7: `STATUS_PARKED` (Linux)

Constantes de prioridade do processo

psutil.CLASSE_PRIORIDADE_EM_TEMPO_REAL

psutil. CLASSE_DE_ALTA_PRIORIDADE

psutil. CLASSE DE PRIORIDADE ACIMA DO NORMAL

psutil. CLASSE_DE_PRIORIDADE_NORMAL

psutil. IDLE_PRIORITY_CLASS

psutil. CLASSE DE PRIORIDADE ABAIXO DO NORMAL

Representam a prioridade de um processo no Windows (consulte SetPriorityClass). Podem ser usados em conjunto com outras funções `psutil.Process.nice()` para obter ou definir a prioridade de um processo.

Disponibilidade: Windows

psutil. IOPRIO_CLASS_NONE

psutil. IOPRIO_CLASS_RT

psutil. IOPRIO_CLASS_BE

psutil. IOPRIO_CLASS_IDLE

Um conjunto de números inteiros que representam a prioridade de E/S de um processo no Linux. Eles podem ser usados em conjunto com a chamada de sistema `ioprio_get` `psutil.Process.ionice()` para obter ou definir a prioridade de E/S do processo. `*IOPRIO_CLASS_NONE*` e `*IOPRIO_CLASS_BE*` (melhor esforço) são os valores padrão para qualquer processo que não tenha definido uma prioridade de E/S específica. `*IOPRIO_CLASS_RT*` (tempo real) significa que o processo terá acesso prioritário ao disco, independentemente do que mais estiver acontecendo no sistema. `*IOPRIO_CLASS_IDLE*` significa que o processo receberá tempo de E/S quando nenhum outro processo precisar do disco. Para mais informações, consulte os manuais do utilitário de linha de comando `ionice` ou da chamada de sistema ` ioprio_get` .

Disponibilidade: Linux

psutil. IOPRIO_VERYLOW

psutil. IOPRIO_LOW

psutil. IOPRIO_NORMAL

psutil. IOPRIO_HIGH

Um conjunto de números inteiros que representam a prioridade de E/S de um processo no Windows. Eles podem ser usados em conjunto com outras funções `psutil.Process.ionice()` para obter ou definir a prioridade de E/S do processo.

Disponibilidade: Windows

Adicionado na versão 5.6.2.

Constantes de recursos do processo

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

psutil. RLIM_INFINITY

psutil. RLIMIT_AS

psutil. RLIMIT_CORE

psutil. RLIMIT_CPU

psutil. RLIMIT_DATA

psutil. RLIMIT_FSIZE

psutil. RLIMIT_MEMLOCK

psutil. RLIMIT_NOFILE

psutil. RLIMIT_NPROC

psutil. RLIMIT_RSS

psutil. RLIMIT_STACK

Específico para Linux:

psutil.RLIMIT_LOCKS

psutil.RLIMIT_MSGQUEUE

psutil.RLIMIT_NICE

psutil.RLIMIT_RTPRIO

psutil.RLIMIT_RTTIME

psutil.RLIMIT_SIGPENDING

FreeBSD specific:

psutil.RLIMIT_SWAP

psutil.RLIMIT_SBSIZE

psutil.RLIMIT_NPTS

Constants used for getting and setting process resource limits to be used in conjunction with `psutil.Process.rlimit()`. See resource.getrlimit for further information.

Availability: Linux, FreeBSD

Changed in version 5.7.3: added FreeBSD support, added `RLIMIT_SWAP`, `RLIMIT_SBSIZE`, `RLIMIT_NPTS`.

Connections constants

psutil.CONN_ESTABLISHED

psutil.CONN_SYN_SENT

psutil.CONN_SYN_RECV

psutil.CONN_FIN_WAIT1

psutil.CONN_FIN_WAIT2

psutil.CONN_TIME_WAIT

psutil.CONN_CLOSE

psutil.CONN_CLOSE_WAIT

psutil.CONN_LAST_ACK

psutil.CONN_LISTEN

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

psutil.CONN_NONE

psutil.CONN_DELETE_TCB (*Windows*)

psutil.CONN_IDLE (*Solaris*)

psutil.CONN_BOUND (*Solaris*)

A set of strings representing the status of a TCP connection. Returned by `psutil.Process.net_connections()` and `psutil.net_connections()` (*status* field).

Hardware constants

psutil.AF_LINK

Constant which identifies a MAC address associated with a network interface. To be used in conjunction with `psutil.net_if_addrs()`.

Added in version 3.0.0.

psutil.NIC_DUPLEX_FULL

psutil.NIC_DUPLEX_HALF

psutil.NIC_DUPLEX_UNKNOWN

Constants which identifies whether a NIC (network interface card) has full or half mode speed. NIC_DUPLEX_FULL means the NIC is able to send and receive data (files) simultaneously, NIC_DUPLEX_FULL means the NIC can either send or receive data at a time. To be used in conjunction with `psutil.net_if_stats()`.

Added in version 3.0.0.

psutil.POWER_TIME_UNKNOWN

psutil.POWER_TIME_UNLIMITED

Whether the remaining time of the battery cannot be determined or is unlimited. May be assigned to `psutil.sensors_battery()`'s *secsleft* field.

Added in version 5.1.0.

psutil.version_info

A tuple to check psutil installed version. Example:

```
>>> import psutil
>>> if psutil.version_info >= (4, 5):
...     pass
```

Recipes

Find process by name

Check string against `Process.name()`:

```
import psutil

def find_procs_by_name(name):
    "Return a list of processes matching 'name'."
    ls = []
    for p in psutil.process_iter(['name']):
        if p.info['name'] == name:
            ls.append(p)
    return ls
```

A bit more advanced, check string against `Process.name()`, `Process.exe()` and `Process.cmdline()`:

```
import os
import psutil

def find_procs_by_name(name):
    "Return a list of processes matching 'name'."
    ls = []
    for p in psutil.process_iter(["name", "exe", "cmdline"]):
        if name == p.info['name'] or \
                p.info['exe'] and os.path.basename(p.info['exe']) == name or \
                p.info['cmdline'] and p.info['cmdline'][0] == name:
            ls.append(p)
    return ls
```

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Kill process tree

```
import os
import signal
import psutil

def kill_proc_tree(pid, sig=signal.SIGTERM, include_parent=True,
                   timeout=None, on_terminate=None):
    """Kill a process tree (including grandchildren) with signal
    "sig" and return a (gone, still_alive) tuple.
    "on_terminate", if specified, is a callback function which is
    called as soon as a child terminates.
    """
    assert pid != os.getpid(), "won't kill myself"
    parent = psutil.Process(pid)
    children = parent.children(recursive=True)
    if include_parent:
        children.append(parent)
    for p in children:
        try:
            p.send_signal(sig)
        except psutil.NoSuchProcess:
            pass
    gone, alive = psutil.wait_procs(children, timeout=timeout,
                                    callback=on_terminate)
    return (gone, alive)
```

Filtering and sorting processes

A collection of code samples showing how to use `process_iter()` to filter processes and sort them. Setup:

```
>>> import psutil
>>> from pprint import pprint as pp
```

Processes owned by user:

```
>>> import getpass
>>> pp([(p.pid, p.info['name']) for p in psutil.process_iter(['name', 'username']) if p.info['username'] == getpass.getuser()])
(16832, 'bash'),
(19772, 'ssh'),
(20492, 'python')]
```

Processes actively running:

```
>>> pp([(p.pid, p.info) for p in psutil.process_iter(['name', 'status']) if p.info['status'] == psutil.STATUS_RUNNING])
[(1150, {'name': 'Xorg', 'status': 'running'}),
 (1776, {'name': 'unity-panel-service', 'status': 'running'}),
 (20492, {'name': 'python', 'status': 'running'})]
```

Processes using log files:

```
>>> for p in psutil.process_iter(['name', 'open_files']):
...      for file in p.info['open_files'] or []:
...          if file.path.endswith('.log'):
...              print("%-5s %-10s %s" % (p.pid, p.info['name'][:10], file.path))
...
1510  upstart    /home/giampaolo/.cache/upstart/unity-settings-daemon.log
2174  nautilus   /home/giampaolo/.local/share/gvfs-metadata/home-ce08efac.log
2650  chrome     /home/giampaolo/.config/google-chrome/Default/data_reduction_proxy_leveldb/000003.log
```

Processes consuming more than 500M of memory:

```
>>> pp([(p.pid, p.info['name'], p.info['memory_info'].rss) for p in psutil.process_iter(['name', 'memory_info']) if p.info['memory_info'].rss > 500 * 1024 * 1024])
[(2650, 'chrome', 532324352),
 (3038, 'chrome', 1120088064),
 (21915, 'sublime_text', 615407616)]
```

Top 3 processes which consumed the most CPU time:

```
>>> pp([(p.pid, p.info['name'], sum(p.info['cpu_times'])) for p in sorted(psutil.process_iter(['name', 'cpu_times']), key=lambda p: sum(p.info['cpu_times'][:2]))]
[-3:])
[(2721, 'chrome', 10219.73),
 (1150, 'Xorg', 11116.989999999998),
 (2650, 'chrome', 18451.97)]
```

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Bytes conversion

```
import psutil

def bytes2human(n):
    # http://code.activestate.com/recipes/578019
    # >>> bytes2human(10000)
    # '9.8K'
    # >>> bytes2human(100001221)
    # '95.4M'
    symbols = ('K', 'M', 'G', 'T', 'P', 'E', 'Z', 'Y')
    prefix = {}
    for i, s in enumerate(symbols):
        prefix[s] = 1 << (i + 1) * 10
    for s in reversed(symbols):
        if abs(n) >= prefix[s]:
            value = float(n) / prefix[s]
            return '%.1f%s' % (value, s)
    return "%sB" % n

total = psutil.disk_usage('/').total
print(total)
print(bytes2human(total))
```

...prints:

```
100399730688
93.5G
```

FAQs

- Q: Why do I get `AccessDenied` for certain processes?
- A: This may happen when you query processes owned by another user, especially on macOS (see issue #883) and Windows. Unfortunately there's not much you can do about this except running the Python process with higher privileges. On Unix you may run the Python process as root or use the SUID bit (`ps` and `netstat` does this). On Windows you may run the Python process as NT AUTHORITY\SYSTEM or install the Python script as a Windows service (ProcessHacker does this).
- Q: is MinGW supported on Windows?
- A: no, you should Visual Studio (see development guide).

Running tests

```
$ make test
```

Debug mode

If you want to debug unusual situations or want to report a bug, it may be useful to enable debug mode via `PSUTIL_DEBUG` environment variable. In this mode, psutil may (or may not) print additional information to stderr. Usually these are error conditions which are not severe, and hence are ignored (instead of crashing). Unit tests automatically run with debug mode enabled. On UNIX:

```
$ PSUTIL_DEBUG=1 python3 script.py
psutil-debug [psutil/_psutil_linux.c:150]> setmntent() failed (ignored)
```

On Windows:

```
set PSUTIL_DEBUG=1 python.exe script.py
psutil-debug [psutil/arch/windows/proc.c:90]> NtWow64ReadVirtualMemory64(pbi64.PebBaseAddress) -> 998 (Unknown error) (ignored)
```

Python 2.7

Latest version spporting Python 2.7 is psutil 6.1.1. The 6.1.X serie may receive critical bug-fixes but no new features. It will be maintained in the dedicated python2 branch. To install it:

```
$ python2 -m pip install psutil==6.1.*
```

Security

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

Development guide

If you want to develop psutil take a look at the DEVGUIDE.rst.

Platforms support history

- psutil 7.2.0 (2025-12): publish wheels for Linux musl
- psutil 7.1.2 (2025-10): publish wheels for free-threaded Python
- psutil 7.1.2 (2025-10): no longer publish wheels for 32-bit Python (**Linux** and **Windows**)
- psutil 7.1.1 (2025-10): drop **SunOS 10**
- psutil 7.1.0 (2025-09): drop **FreeBSD 8**
- psutil 7.0.0 (2025-02): drop Python 2.7
- psutil 5.9.6 (2023-10): drop Python 3.4 and 3.5
- psutil 5.9.1 (2022-05): drop Python 2.6
- psutil 5.9.0 (2021-12): add **MidnightBSD**
- psutil 5.8.0 (2020-12): add **PyPy 2** on Windows
- psutil 5.7.1 (2020-07): add **Windows Nano**
- psutil 5.7.0 (2020-02): drop Windows XP & Windows Server 2003
- psutil 5.7.0 (2020-02): add **PyPy 3** on Windows
- psutil 5.4.0 (2017-11): add **AIX**
- psutil 3.4.1 (2016-01): add **NetBSD**
- psutil 3.3.0 (2015-11): add **OpenBSD**
- psutil 1.0.0 (2013-07): add **Solaris**
- psutil 0.1.1 (2009-03): add **FreeBSD**
- psutil 0.1.0 (2009-01): add **Linux, Windows, macOS**

Supported Python versions at the time of writing are cPython 2.7, 3.6+ and PyPy3.

Timeline

- 2026-01-28: 7.2.2 - what's new - diff
- 2025-12-29: 7.2.1 - what's new - diff
- 2025-12-23: 7.2.0 - what's new - diff
- 2025-11-02: 7.1.3 - what's new - diff
- 2025-10-25: 7.1.2 - what's new - diff
- 2025-10-19: 7.1.1 - what's new - diff
- 2025-09-17: 7.1.0 - what's new - diff
- 2025-02-13: 7.0.0 - what's new - diff
- 2024-12-19: 6.1.1 - what's new - diff
- 2024-10-17: 6.1.0 - what's new - diff
- 2024-06-18: 6.0.0 - what's new - diff
- 2024-01-19: 5.9.8 - what's new - diff
- 2023-12-17: 5.9.7 - what's new - diff
- 2023-10-15: 5.9.6 - what's new - diff
- 2023-04-17: 5.9.5 - what's new - diff
- 2022-11-07: 5.9.4 - what's new - diff
- 2022-10-18: 5.9.3 - what's new - diff
- 2022-09-04: 5.9.2 - what's new - diff
- 2022-05-20: 5.9.1 - what's new - diff
- 2021-12-29: 5.9.0 - what's new - diff
- 2020-12-19: 5.8.0 - what's new - diff
- 2020-10-24: 5.7.3 - what's new - diff
- 2020-07-15: 5.7.2 - what's new - diff
- 2020-07-15: 5.7.1 - what's new - diff
- 2020-02-18: 5.7.0 - what's new - diff
- 2019-11-26: 5.6.7 - what's new - diff
- 2019-11-25: 5.6.6 - what's new - diff
- 2019-11-06: 5.6.5 - what's new - diff
- 2019-11-04: 5.6.4 - what's new - diff
- 2019-06-11: 5.6.3 - what's new - diff
- 2019-04-26: 5.6.2 - what's new - diff

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

- 2019-02-15: 5.5.1 - what's new - diff
- 2019-01-23: 5.5.0 - what's new - diff
- 2018-10-30: 5.4.8 - what's new - diff
- 2018-08-14: 5.4.7 - what's new - diff
- 2018-06-07: 5.4.6 - what's new - diff
- 2018-04-13: 5.4.5 - what's new - diff
- 2018-04-13: 5.4.4 - what's new - diff
- 2018-01-01: 5.4.3 - what's new - diff
- 2017-12-07: 5.4.2 - what's new - diff
- 2017-11-08: 5.4.1 - what's new - diff
- 2017-10-12: 5.4.0 - what's new - diff
- 2017-09-10: 5.3.1 - what's new - diff
- 2017-09-01: 5.3.0 - what's new - diff
- 2017-04-10: 5.2.2 - what's new - diff
- 2017-03-24: 5.2.1 - what's new - diff
- 2017-03-05: 5.2.0 - what's new - diff
- 2017-02-07: 5.1.3 - what's new - diff
- 2017-02-03: 5.1.2 - what's new - diff
- 2017-02-03: 5.1.1 - what's new - diff
- 2017-02-01: 5.1.0 - what's new - diff
- 2016-12-21: 5.0.1 - what's new - diff
- 2016-11-06: 5.0.0 - what's new - diff
- 2016-10-05: 4.4.2 - what's new - diff
- 2016-10-25: 4.4.1 - what's new - diff
- 2016-10-23: 4.4.0 - what's new - diff
- 2016-09-01: 4.3.1 - what's new - diff
- 2016-06-18: 4.3.0 - what's new - diff
- 2016-05-14: 4.2.0 - what's new - diff
- 2016-03-12: 4.1.0 - what's new - diff
- 2016-02-17: 4.0.0 - what's new - diff
- 2016-01-20: 3.4.2 - what's new - diff
- 2016-01-15: 3.4.1 - what's new - diff
- 2015-11-25: 3.3.0 - what's new - diff
- 2015-10-04: 3.2.2 - what's new - diff
- 2015-09-03: 3.2.1 - what's new - diff
- 2015-09-02: 3.2.0 - what's new - diff
- 2015-07-15: 3.1.1 - what's new - diff
- 2015-07-15: 3.1.0 - what's new - diff
- 2015-06-18: 3.0.1 - what's new - diff
- 2015-06-13: 3.0.0 - what's new - diff
- 2015-02-02: 2.2.1 - what's new - diff
- 2015-01-06: 2.2.0 - what's new - diff
- 2014-09-26: 2.1.3 - what's new - diff
- 2014-09-21: 2.1.2 - what's new - diff
- 2014-04-30: 2.1.1 - what's new - diff
- 2014-04-08: 2.1.0 - what's new - diff
- 2014-03-10: 2.0.0 - what's new - diff
- 2013-11-25: 1.2.1 - what's new - diff
- 2013-11-20: 1.2.0 - what's new - diff
- 2013-10-22: 1.1.2 - what's new - diff
- 2013-10-08: 1.1.1 - what's new - diff
- 2013-09-28: 1.1.0 - what's new - diff
- 2013-07-12: 1.0.1 - novidades - diferenças
- 10/07/2013: 1.0.0 - novidades - diferenças
- 03/05/2013: 0.7.1 - novidades - diferença
- 12/04/2013: 0.7.0 - novidades - diferenças
- 16/08/2012: 0.6.1 - novidades - diferenças
- 13/08/2012: 0.6.0 - novidades - diferença
- 29/06/2012: 0.5.1 - novidades - diferenças
- 27/06/2012: 0.5.0 - novidades - diferença
- 14/12/2011: 0.4.1 - novidades - diferenças
- 29/10/2011: 0.4.0 - novidades - diferenças
- 2011-07-08: 0.3.0 - novidades - diff
- 20/03/2011: 0.2.1 - novidades - diferenças
- 13/11/2010: 0.2.0 - novidades - diferença

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio

- 06/03/2009: 0.1.1 - novidades - diferenças
- 27/01/2009: 0.1.0 - novidades - diferenças

estável

Monetize seu público: financie um projeto ou site de código aberto com a EthicalAds, **uma rede de anúncios que prioriza a privacidade.**

Anúncios da EthicalAds

Fechar anúncio